BZ205 Veri Yapıları ve Algoritmalar

Vize Konuları – Detaylı Konu Anlatımı

Ders videoları (transkriptler), slaytlar, çıkmış sorular ve öğrenci notlarından derlenmiştir

27 Ekim 2026

Nasıl kullanılır?

Kırmızı çerçeveli **"Hoca derste vurguladı"** kutuları ders videolarından birebir alıntılardır; köşeli parantezdeki [video dakika] ile videoya dönüp bakabilirsiniz. Turuncu kutular sınavda sık yapılan hataları, mavi kutular ek bilgileri içerir. Konular dersteki sırayla verilmiştir; vize kapsamı genelde 1–8. bölümlerdir, 9. bölüm bazı yıllarda vizeye girmiştir.

İçindekiler

1 Akış Diyagramları **2**

1.1 Algoritmanın üç gösterimi . . . 2
1.2 Standart semboller (derste kullanılan) . . . 2
1.3 Kontrol yapıları . . . 3
1.4 Derste çözülen örnekler . . . 3
1.5 Akış diyagramı çizerken kurallar . . . 4
1.6 Sınavda istenen akış diyagramları ve nerede oldukları . . . 4

2 Karmaşıklık Analizi **6**

2.1 Tanım ve amaç . . . 6
2.2 Dört terim . . . 6
2.3 Asimptotik notasyonlar . . . 6
2.4 Bilinen büyüme fonksiyonları . . . 7
2.5 Sayma kuralları . . . 7
2.6 Derste çözülen örnekler . . . 7
2.7 Sınavda yürütme zamanı nasıl yazılır? (şablon) . . . 10

3 Sıralama Algoritmaları **12**

3.1 Araya sokma sıralama (Insertion sort) . . . 12
3.2 Seçmeli sıralama (Selection sort) . . . 14
3.3 Kabarcık sıralama (Bubble sort) . . . 16
3.4 Birleşmeli sıralama (Merge sort) . . . 18
3.5 Kümeleme sıralama (Heap sort) . . . 21

3.6 Hızlı sıralama (Quick sort) . . . 26
3.7 Radiks sıralama (Radix sort) . . . 28
3.8 Özet tablo . . . 30

4 Listeler ve Bağlantılı Listeler 31
4.1 Veri yapısı ve soyut veri tipi . . . 31
4.2 Dizi ile bağlantılı liste karşılaştırması . . . 31
4.3 Liste işlemleri (slayt örneği: 34, 12, 52, 16, 12) . . . 31
4.4 Bağlantılı liste gerçekleştirmesi (C++) . . . 31
4.5 Liste çeşitleri . . . 36

5 Yığın (Stack) ve Kuyruk (Queue) 37
5.1 Yığın – LIFO (son giren ilk çıkar) . . . 37
5.2 Postfix (sonek) ifadenin yığınla hesaplanması . . . 38
5.3 Kuyruk – FIFO (ilk giren ilk çıkar) . . . 39
5.4 Farz edin ki sorularında yığın ve kuyruğu bozmadan gezmek . . . 40
5.5 Zaman ve alan karmaşıklığı . . . 42

6 İkili Ağaçlar ve İkili Arama Ağaçları 43
6.1 Neden ağaç? . . . 43
6.2 İfade ağaçları . . . 43
6.3 Ağaç gezme (traversal) . . . 43
6.4 İkili arama ağacı (İAA) . . . 44
6.5 İAA kodları . . . 45
6.6 Zaman ve alan karmaşıklığı . . . 48

7 AVL Ağaçları 49
7.1 Neden AVL? . . . 49
7.2 Denge kuralı . . . 49
7.3 Dört problem ve çözümleri . . . 49
7.4 Ekleme . . . 50
7.5 Silme . . . 53

8 "Farz Edin Ki" Soruları – Çözüm Rehberi 55
8.1 Temel işlemler tablosu . . . 55
8.2 Küme işlemleri . . . 55
8.3 Şablon (hocanın tahtadaki çözümüne göre) . . . 56
8.4 Cevap kâğıdında olması gerekenler . . . 57

9 Ek: B+ Ağaçları ve Graf Arama (bazı yıllarda vizeye girdi) 58
9.1 B+ ağacı . . . 58

9.2 Graflar: BFS, DFS ve sonrakök 60

1 Akış Diyagramları

1.1 Algoritmanın üç gösterimi

Hoca derse menemen tarifi örneğiyle başlar: bir algoritma üç şekilde yazılabilir.

1. **Kaba kod (sözde kod):** Bildiğimiz dilde (Türkçe/İngilizce), yarı konuşma dili yarı programlama dili ile detaya girmeden yazılan adımlar. *"Tavayı koy, yağı ısıt, soğanı doğra…"*
2. **Akış diyagramı:** Algoritmanın *şekilsel* gösterimi. Elektronik devre şeması gibi standart sembolleri vardır; *"bu standartları bilen herhangi bir programcı akış diyagramını gördüğü zaman kodu yazabiliyor olması lazım"*.
3. **Gerçek kod:** Bir programlama diliyle (C, C++, Java…) yazılmış kod.

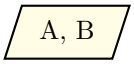
! Hoca derste vurguladı [video 1 Giriş 03:03]

"Sınavlarda program kodu sormuyorum genelde… Ya kaba kodu ya da akış diyagramı şeklinde olacak." Öğrenci aktarımı: *"Hangi dilde yazdığınız fark etmez, kaba kodunu da yazsanız mantığını anlayıp anlamadığınıza bakıyorum."*

1.2 Standart semboller (derste kullanılan)



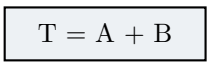
Başla / Dur: yuvarlatılmış dikdörtgen veya elips. Çıkışı varsa başlangıç, girişi varsa bitiş.



Klavyeden giriş: paralelkenar.



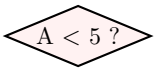
Ekrana çıktı ("göz" sembolü): *"Biz yine göz sembolüne sadık kalacağız"*.



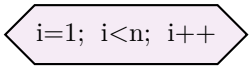
İşlem: dikdörtgen; birden fazla işlem tek kutuda olabilir.



Fonksiyon çağırma: çift kenarlı dikdörtgen. Kendi yazdığınız fonksiyonun akışını ayrı sayfada verin.



Koşul (if / switch): eşkenar dörtgen (baklava dilimi); çıkışlar Evet/Hayır (veya +/−).



for (kompakt gösterim): *"sıra önemli: 1. başlangıç, 2. koşul, 3. artım"*.



Etiket (bağlayıcı): diyagram sayfaya sığmazsa aynı numaralı etiketten devam edilir.

! Hoca derste vurguladı [video 2 Akış Diyagramı 08:46]

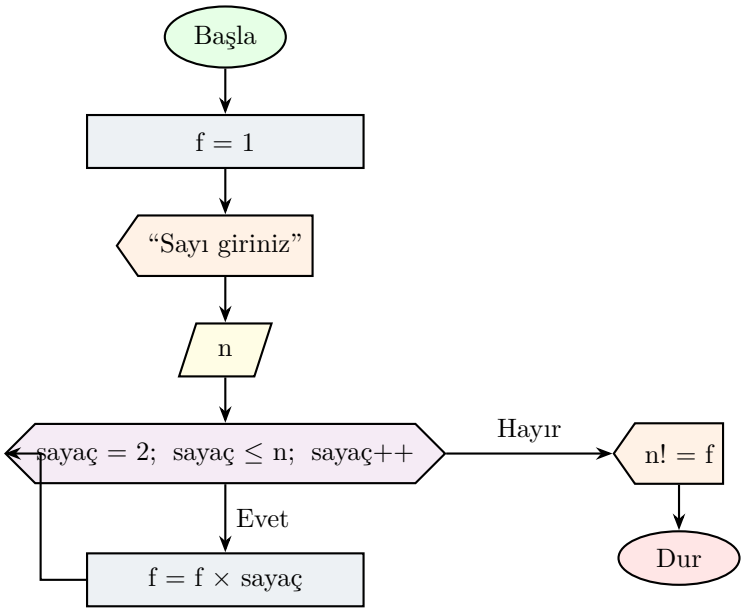
"Ok işareti önemli arkadaşlar. Ok algoritmanın akış yönünü gösteriyor." Döngünün yukarı mı aşağı mı döndüğü, nereden çıkıldığı oklarla belli olmalıdır.

1.3 Kontrol yapıları

- **if–else:** Koşul sağlanırsa bir dal, sağlanmazsa diğer dal; ikisi aşağıda birleşip devam eder. `else` yoksa "Hayır" dalı doğrudan birleşme noktasına gider.
- **İç içe if / switch–case:** Her koşul ayrı karar kutusu; switch'te sırayla kontrol edilir, sonunda **default** olmalıdır (menü örneği: tanımsız tuşa basılırsa uyarı).
- **while:** Önce koşula bakılır; koşul doğruysa döngüye girilir, yanlışsa döngü atlanır (sıfır kez çalışabilir).
- **do–while:** Gövde *en az bir kez* çalışır, sonra koşula bakılır.
- **for:** Başlangıç (bir kez), koşul (her turda), artım (her tur sonunda); akış diyagramında üçü tek **altıgen döngü sembolüne** yazılır, gövdeden altıgene geri ok döner, "Hayır" çıkışı döngünün sonrasına gider. Örnek: `for(i=1; i<n; i++)` n=1 ise hiç girmez; n=10 ise 9 kez girer.
- **İç içe döngüler:** İç döngü bittikten sonra yapılacak bir işlem (X), dış döngüye *dönüş yolunun üzerine* çizilir; while bittikten sonra yapılacak işlem (Y) dıştaki for'a dönüş yoluna konur.

1.4 Derste çözülen örnekler

Faktöriyel (döngü ile)



! Hoca derste vurguladı [video 2 Akış Diyagramı 42:49]

Hoca bu diyagramı test ettirdi: *"0 faktöriyeli bulur mu? 1 faktöriyeli bulabilir miyim?"* Sayaç 2'den başladığı ve f=1 olduğu için n=0 ve n=1'de döngüye girmeden 1 yazılır, yani doğru. **Kendi diyagramınızı uç değerlerle test edin.** Ayrıca koddaki "Sayı giriniz" mesajı için ekran sembolü eklenmesi gerektiğini hatırlattı.

Diğer derste çözülen örnekler: iki sayının aritmetik ortalaması (iki ayrı ya da tek giriş kutusu olabilir), $ax^2 + bx + c$ köklerinin bulunması (delta > 0, $= 0$, < 0 dalları ve sayfa sığmayınca etiket kullanımı), iki matrisin toplamı (iç içe iki for: satır ve sütun).

Hocanın verdiği ödev / olası sınav soruları

"Bağlantılı listeye verilen bir veriyi ekleyen fonksiyonun akış diyagramı" ve *"bir ikili ağaca verilen bir sayıyı arayan fonksiyonun akış diyagramı"* [video 2 Akış Diyagramı 55:51]. Sınavlarda: hızlı sıralama (2024, 2025, bilinmeyen yıl), kümeleme sıralama (2020 büt, 2021 final), birleşmeli sıralama (final), İAA silme (2025), bağlantılı listeye ekleme (bilinmeyen yıl), B+ ekleme ve BFS (2025 final), her farz edin ki sorusu.

1.5 Akış diyagramı çizerken kurallar

1. Her diyagramın bir **başla** ve en az bir **dur** sembolü olmalı (küçük bir fonksiyon bile olsa).
2. Akış diyagramı *koddan önce* tasarlanır. *"Kodu bir web sitesine döküp akış diyagramını çıkartırıyorsunuz ki bu tamamen yanlış"* [video 2 07:43].
3. *"Tek doğru akış diyagramı budur diye bir kural yok. Mantık doğru olduğu sürece şekilsel olarak farklı olabilir"* [video 2 14:32].
4. Fonksiyon çağrıları çift kenarlı kutu; çağrılan fonksiyon (ör. `Bolumle`, `Ara`) ayrıca çizilir.
5. Döngünün dönüş oku koşul kutusuna geri gitmeli; çıkış dalı açıkça belirtilmeli.
6. Sıralama algoritmalarının slaytlarda akış diyagramı yoktur ama *sınavda sorulur*: *"hiçbir sort'un akış diyagramı yok ama soruyor"* (öğrenci). Kaba koddan akış diyagramına çevirme pratiği yapın.

1.6 Sınavda istenen akış diyagramları ve nerede oldukları

Vizelerde doğrudan sorulan veya farz edin ki sorularının içinde kullanılan bütün akış diyagramları, kaba kodları ve karmaşıklıklarıyla birlikte **ait oldukları konunun içinde**, algoritmanın anlatıldığı yerde verilmiştir (ör. kabarcık sıralamanın akış diyagramı Sıralama bölümündeki kabarcık sıralama başlığındadır). Diyagramlar derste gösterilen standart sembollerle çizilmiştir: **elips** başla/dur, **paralelkenar** giriş, **göz sembolü** ekrana yazma, **dikdörtgen** işlem, **çift kenarlı dikdörtgen** fonksiyon çağrısı, **baklava** koşul. `for` döngüleri derste gösterilen **altıgen döngü sembolüyle** çizilmiştir: başlangıç, koşul ve artım tek kutuda yazılır (`i = 0; i < n; i++`), gövdenin sonundan altıgene geri ok döner, döngü bitince altıgenin "Hayır" çıkışından devam edilir. `while` döngüleri baklava (koşul) ile gösterilir.

Diyagram	Sınavda istendiği yer	Bulunduğu konu
Hızlı sıralama + bölümleme	2024–25 vize, 2025–26 vize, yılı bilinmeyen vize	3 Sıralama
Kümeleme sıralama	2020 büt, 2021 final	3 Sıralama
Birleşmeli sıralama	bir final	3 Sıralama
Radiks sıralama	"muhtemel vize" listesi; öğrenci: "radixin kodunu da istemişti"	3 Sıralama
Kabarcık, seçmeli, araya sokma	hocanın derste verdiği örnek: "kabarcık sıralamanın akış diyagramını çiziniz diyebiliriz"	3 Sıralama
Doğrusal / ikili arama, en küçük eleman	derste analiz edilen örnekler	2 Karmaşıklık
Bağlantılı listeye ekleme, silme, gezme	yılı bilinmeyen vize; hocanın derste verdiği ödev	4 Listeler
Push/Pop, Enqueue/Dequeue, postfix, bozmadan gezme	2021 vizesi (postfix); farz edin ki soruları	5 Yığın ve kuyruk
İkili ağaçta arama, İAA ekleme/silme, gezmeler	hocanın ödevi (video 2, 55:51); 2025–26 vize (silme, sonrakök)	6 Ağaçlar
AVL ağacına ekleme	AVL sorularının kodu	7 AVL
Farz edin ki fonksiyonu	bütün vize, final ve bütlerde (her seferinde)	8 Farz edin ki
BFS, DFS, B+ ekleme	2025–26 final	9 B+ ve graf

! Hoca derste vurguladı [video 2 Akış Diyagramı 16:04 ve 14:32]

"Bir kod verdik veya böyle bir sözel bir ifade verdim, bunun akış diyagramını çizin diyebiliriz veya işte kabarcık sıralamanın akış diyagramını çiziniz diyebiliriz... Mantığı çözmemiz lazım."
"Tek doğru akış diyagramı budur diye bir kural yok. Sizinki de mantık doğru olduğu sürece farklı şekiller kullanabilirsiniz."

Ezberlemeden çizme yöntemi

1. Önce kaba kodu yaz (satır satır).
2. Her atama/işlem satırı → dikdörtgen; ardışık atamalar tek kutuda birleşebilir.
3. Her `if`/`while` koşulu → baklava; her `for` → altıgen döngü sembolü (başlangıç; koşul; artım).
4. Döngü gövdesinin sonundan koşula *ok ile* dön; "Hayır" çıkışını döngüden sonraki ilk kutuya bağla.
5. Fonksiyon çağrıları (özyineleme dahil) çift kenarlı kutu; o fonksiyonun diyagramını ayrıca çiz.

2 Karmaşıklık Analizi

2.1 Tanım ve amaç

Karmaşıklık analizi, bir algoritmanın *kodlanmadan önce* davranışının teorik olarak ortaya konmasıdır; genelde kaba koddan veya akış diyagramından yapılır. Aynı işi yapan algoritmaları karşılaştırmak (hız veya bellek açısından) için kullanılır. Kodlayıp ölçmek hem zaman alır hem de yanıltıcıdır: farklı makine, RAM, programlama dili ve programcı tecrübesi sonucu etkiler (*"fizikteki eşit koşullarda deney gibi"*).

2.2 Dört terim

Yürütme zamanı $T(n)$	Algoritmanın işini yapması için temel kabul edilen işlemlerden (atama, karşılaştırma, toplama, artırma, return, yazdırma…) *kaç tane* yapılması gerektiğini veren bağıntı. Girdi birden fazla ise $T(n, m)$.
Zaman karmaşıklığı	Asimptotik notasyona göre karmaşıklık *mertebesi* (davranışı): $O(n)$, $O(n^2)$, $O(\log n)$…
Alan maliyeti	Algoritmanın çalışması için gereken bellek miktarı.
Alan karmaşıklığı	Eleman sayısı büyüdükçe bellek gereksiniminin artış mertebesi.

! Hoca derste vurguladı [video 3 Karmaşıklık 06:09]

"Sınavlarda vesaire en çok karıştırdığımız şey bu. Yürütme zamanı dediğimiz zaman neyi istiyoruz? Zaman karmaşıklığı dediğimiz zaman neyi istiyoruz? Verilen cevaplar arasındaki en büyük karmaşa bu. ***Eğer karıştırılırsa bu konuyu bilmiyorsunuz demektir.****"*
Yürütme zamanı bir *fonksiyondur* ($T(n) = 3n + 4$); zaman karmaşıklığı bir *mertebedir* ($O(n)$).

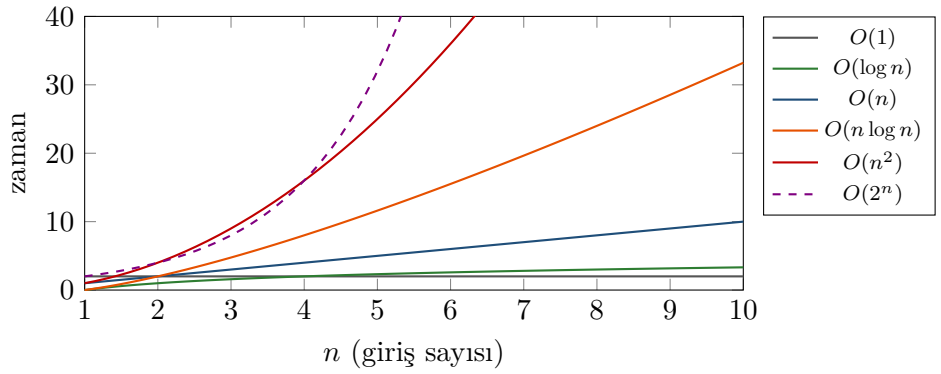
2.3 Asimptotik notasyonlar

- O ("order of") – **en kötü durum**, üst sınır: algoritma bundan daha kötü çalışamaz. *Genelde hesaplanan budur.*
- Ω – **en iyi durum**, alt sınır: bundan daha iyi çalışamaz.
- Θ – **ortalama durum**. Ders kapsamında: *"en iyi ve en kötü durum zaman karmaşıklıklarının derecesi aynıysa ortalama durumdan söz edebiliriz"* (ör. $3n + 1$ ve $4n \Rightarrow \Theta(n)$). Dereceler farklıysa (n ve n^2) Θ verilmez.

! Hoca derste vurguladı [video 3 Karmaşıklık 22:52 ve 41:28]

"Soruda verilen algoritmanın karmaşıklık analizini yapın derse burada en kötü durumu soruyor demektir. İllaki en kötü durum nedir diye sormasına gerek yok."
"Bir kısmınız şöyle yazıyor: $O = O(n^2)$. Öyle bir şey olmaz." Doğrusu: $T(n) = 3n^2 + \ldots$, zaman karmaşıklığı $O(n^2)$.

2.4 Bilinen büyüme fonksiyonları



Mertebe	Nerede karşımıza çıkar
$O(1)$ sabit	ekrana tek değer yazma, dizide k. elemana erişim (findKth), yığına push/pop, bağlantılı listede başa ekle
$O(\log n)$	ikili arama ($n = 2^h \Rightarrow h = \log_2 n$), her turda indisi yarıya bölen döngü, dengeli ağaçta arama
$O(n)$	tek döngü (1'den n'e), doğrusal arama, listeyi yazdırma
$O(n \log n)$	birleşmeli, kümeleme, (iyi durumda) hızlı sıralama
$O(n^2)$	iç içe iki döngü; kabarcık, seçmeli, araya sokma; hızlı sıralama en kötü
$O(n^3)$	üç iç içe döngü; `for i=1..n`² içinde `for j=1..n`
$O(2^n)$, $O(n!)$	bazı özyinelemeli (recursive) fonksiyonlar

Dikkat

"For görünce direkt doğrusalı veya kareseli yapıştırmamak lazım" – döngünün kaçtan kaça gittiğine bakın: `for(i=1;i<=n*n;i++) for(j=1;j<=n;j++)` $\Rightarrow n^3$; iç döngü `j=1..1` ise doğrusal; indis her turda ikiye bölünüyorsa logaritmik.

2.5 Sayma kuralları

1. **Değişken tanımları/ilk atamalar ihmal edilir** ve bu yazılır: "değişken tanımlamaları ihmal edilmiştir". İhmal edilmezse sadece sabit terim değişir.
2. **for(k=0; k<n; k++):** atama 1 kez, kontrol $n+1$ kez ($k = 0..n$), artırma n kez $\Rightarrow 2n+2$. Gövdeye n kez girilir.
3. **for(k=1; k<n; k++):** atama 1, kontrol n, artırma $n-1 \Rightarrow 2n$. Gövdeye $n-1$ kez girilir.
4. İç döngü maliyeti, dış döngünün kaç kez çalıştırdığıyla *çarpılır*. İç döngünün sınırı dış indise bağlıysa ($k < i$) toplam alınır: $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$.
5. if kontrolü her zaman sayılır; if'in *içi* en kötü durumda her seferinde, en iyi durumda hiç çalışmayabilir.
6. Fonksiyon içinden başka fonksiyon çağrılıyorsa onun maliyeti ayrıca eklenir; return 1 işlemdir.

2.6 Derste çözülen örnekler

1) Dizinin ortalaması

```
float ortalama(int A[], int n) {
  float ort, toplam = 0;   // ihmal
  int k;                   // ihmal
  for (k = 0; k < n; k++)
     toplam += A[k];
  ort = toplam / n;
  return ort;
}
```

Satır	Tekrar
for $(1 + (n+1) + n)$	$2n + 2$
`toplam += A[k]`	n
`ort = toplam/n`	1
`return`	1
$T(n)$	$\mathbf{3n + 4}$

En iyi = en kötü $\Rightarrow O(n)$, $\Omega(n)$, $\Theta(n)$. Grafikte: $n = 0$'da 4 birimden başlayan eğimi 3 olan doğru.

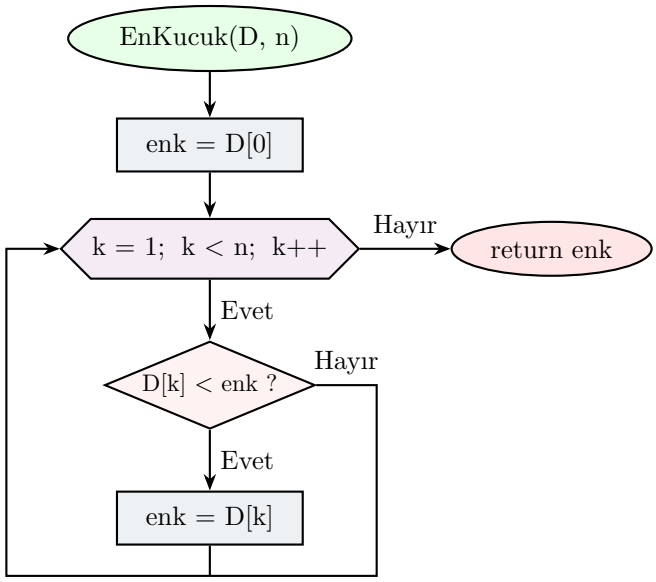
2) Dizideki en küçük eleman

```
int enkucuk(int A[], int n) {
  int enk, k;              // ihmal
  enk = A[0];
  for (k = 1; k < n; k++)
     if (A[k] < enk)
        enk = A[k];
  return enk;
}
```

Satır	En kötü	En iyi
`enk=A[0]`	1	1
for	$2n$	$2n$
if	$n - 1$	$n - 1$
`enk=A[k]`	$n - 1$	0
return	1	1
$T(n)$	$\mathbf{4n}$	$\mathbf{3n + 1}$

En kötü: dizi *büyükten küçüğe* sıralı (if içine her seferinde girer). En iyi: dizi *küçükten büyüğe* sıralı veya tüm elemanlar eşit. $O(n)$, $\Omega(n)$, $\Theta(n)$.

Akış diyagramı: en küçük eleman



En büyük eleman için < yerine > yazılır.

3) İki matrisin toplamı ($n \times m$)

```
for (i = 0; i < n; i++)          // 2n + 2
   for (j = 0; j < m; j++)       // n(2m + 2)
      C[i][j] = A[i][j] + B[i][j];   // n*m
```

$T(n,m) = 2n+2+2nm+2n+nm = \mathbf{3nm+4n+2}$. En kötü durum kare matris ($n=m$): $O(n^2)$ (karesel). En iyi durum *sütun vektörü* ($m=1$): $T = 7n+2 \Rightarrow \Omega(n)$. Dereceler farklı $\Rightarrow \Theta$ yok.

! Hoca derste vurguladı [video 4 Sıralama 1 34:21]

"İllaki sadece koşullu durumlarda en iyi durum oluşacak diye bir şey yok. Verinin geliş şekli farklı olursa da sonuç farklı olur."

! Hoca derste vurguladı Toplama + atama tek işlem mi? [video 4 Sıralama 1 25:32]

"Dışarıdan giriyorum. Buradan n kez çalışır. Dıştaki fordan. İçerideki for da m kez girer. O zaman şuradaki C'nin maliyeti $n \cdot m$."

`C[i][j] = A[i][j] + B[i][j]` satırında toplama *ve* atama olmasına rağmen hoca satırı **tek işlem** sayar (nm, $2nm$ değil); sonuç bu yüzden $3nm+4n+2$'dir. Aritmetik ortalamadaki `toplam += A[k]` de aynı şekilde n sayılır. Kural: **basit ifade satırı = 1 birim** × çalışma sayısı; yalnızca `for` başlığı 3 parçaya ayrılır (atama 1, kontrol $n+1$, artırma n). Ayrı sayılsa bile yalnız katsayı değişir ($4nm$), derece ve $O(nm)$ aynı kalır.

4) Özyinelemeli (recursive) faktöriyel

```
int fakt(int n) {
   if (n <= 1)            // n kez kontrol
      return 1;           // 1 kez
   else
      return n * fakt(n-1);   // n-1 kez
}
```

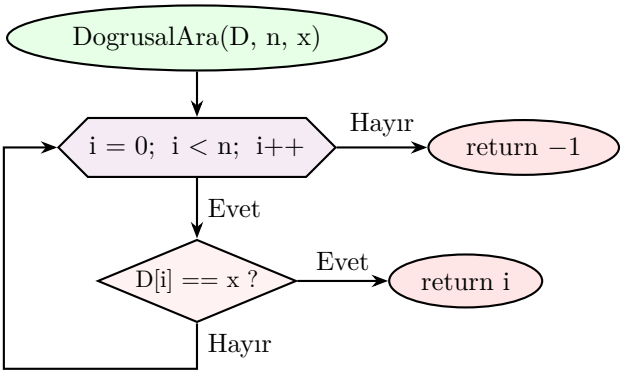
$T(n) = n+1+(n-1) = \mathbf{2n} \Rightarrow O(n)$. Döngülü sürümle aynı zaman karmaşıklığı; fakat **alan maliyeti yüksek**: her çağrı fonksiyonun yeni bir kopyasını oluşturur (5! için 4–5 kopya). Aynı durum birleşmeli ve hızlı sıralama için de geçerlidir.

5) Doğrusal (sıralı) arama

```
DogrusalAra(D, n, x)
  for (i = 0; i < n; i++)
     if (D[i] == x)
        return i
  return -1          // bulunamadi
```

Sırasız dizide baştan sona bakılır. En kötü durum (aranan sonda veya dizide yok): for $2n+2$, if n, return $1 \Rightarrow T(n) = 3n+3 \Rightarrow O(n)$. En iyi durum (aranan ilk elemanda): sabit sayıda işlem $\Rightarrow \Omega(1)$.

Akış diyagramı



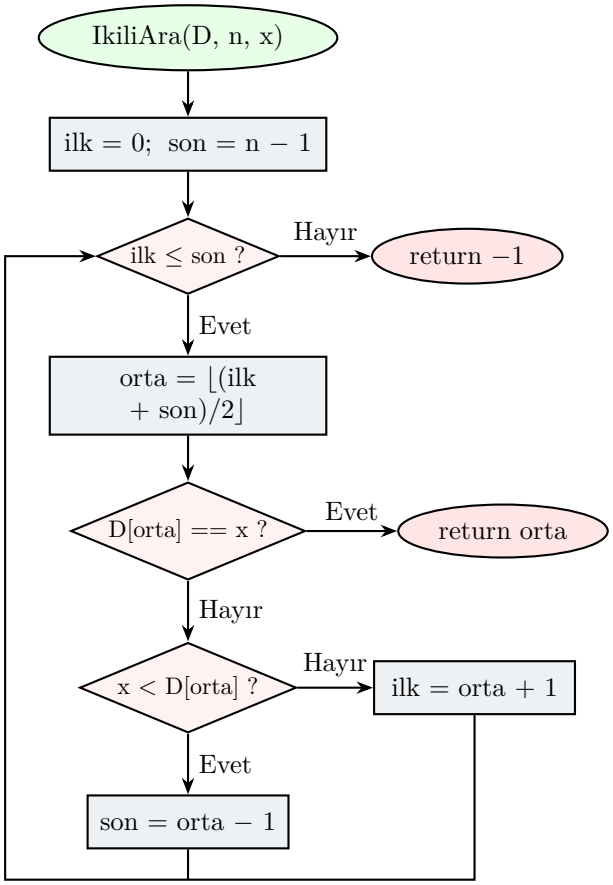
6) İkili arama neden $\log n$?

Sıralı dizide ortadaki eleman ile karşılaştırılır, yarısı elenir. h karşılaştırma sonunda tek eleman kalırsa $n = 2^h \Rightarrow \log_2 n = h$. Dolayısıyla $O(\log n)$. (Verinin sıralanma maliyeti ayrıdır.)

```
IkiliAra(D, n, x)
  ilk = 0;  son = n - 1
  while (ilk <= son)
     orta = (ilk + son) / 2
     if (D[orta] == x) return orta
     if (x < D[orta]) son = orta - 1
     else             ilk = orta + 1
  return -1
```

Her adımda dizinin yarısı elenir (hoca, video 3: 24:34). Dizinin sıralı olması gerekir; sıralama maliyeti ayrıdır.

Akış diyagramı



▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Arama algoritmaları: zaman ve alan karmaşıklığı

Durum	Doğrusal arama	İkili arama	Not
En iyi (Ω)	$\Omega(1)$	$\Omega(1)$	aranan ilk bakılan yerde (ilk eleman / ortadaki)
En kötü (O)	$O(n)$	$O(\log n)$	aranan sonda veya yok / tek eleman kalana kadar yarılama
Ortalama (Θ)	–	–	en iyi ve en kötü derecesi farklı $\Rightarrow \Theta$ verilmez
Ek alan	$O(1)$	$O(1)$	sadece birkaç indis değişkeni (iteratif sürüm)
Ön koşul	yok	dizi sıralı olmalı	sıralama maliyeti ayrıca eklenir

2.7 Sınavda yürütme zamanı nasıl yazılır? (şablon)

1. Kodu/kaba kodu satır numaralarıyla yazın; tanımlamalar için "ihmal" notu düşün.
2. Her satırın yanına kaç kez çalıştığını yazın (tablo). Döngüler için atama/kontrol/artırma ayrı ayrı; gövdedeki basit satırlar (ör. `C[i][j] = A[i][j] + B[i][j]`) tek işlem.
3. İç içe döngülerde çarpın veya ardışık toplam formülünü ($\frac{n(n+1)}{2}$) kullanın.
4. En kötü durum girdisini (ör. ters sıralı dizi) ve en iyi durum girdisini söyleyin; if içini buna göre sayın.

5. Toplayıp $T(n)$'i polinom olarak yazın; en büyük dereceli terim $\Rightarrow O(\cdot)$ / $\Omega(\cdot)$; dereceler eşitse $\Theta(\cdot)$.

Mazeret sınavında sorulan kod tipi

2024–25 mazeret sınavında "aşağıdaki fonksiyonların yürütme zamanlarını ve zaman karmaşıklıklarını hesaplayınız" diye `while` içinde iç içe `for` (`k<i`) ve `i<n*n` döngüleri verildi. Çözümü: `02_Vize_Cozumleri.pdf`, $T(n) = \frac{3}{2}n^3 + \frac{7}{2}n^2 + 4n + 2$ ve $T(n) = \frac{3}{2}n^4 + \frac{5}{2}n^2 + 3n + 4$.

3 Sıralama Algoritmaları

Ders kapsamındaki algoritmalar: araya sokma (insertion), seçmeli (selection), kabarcık (bubble), birleşmeli (merge), kümeleme (heap), hızlı (quick) ve radiks (radix). Hepsinin çalışma mantığı farklıdır; her birinin iyi ve kötü çalıştığı durumlar *en iyi / en kötü durum* analiziyle belirlenir.

! Hoca derste vurguladı [video 4 Sıralama 1 56:17]

*"**Sınavlarda genelde alfabe üzerinden soruyorum. Alfabeyi öğrenelim.**"* [video 8 Sıralama 5 22:15]: Türkçe ve İngilizce harfler karışık gelebilir; birleşik sıra:

A B C Ç D E F G Ğ H I İ J K L M N O Ö P Q R S Ş T U Ü V W X Y Z

"Bir kısmınız bunlar hem İngilizce hem Türkçe olduğu için sıralanamaz diye sınav kağıtlarında yazıyor" – yanlış. *"Programa soktum, Ğ, Ç, Ö, Ü'yü Z'den sonra gösterdi"* – *"böyle olmaz"*.

Örneklerde hocanın derste kullandığı dizi kullanılmıştır: **7, 3, 5, 8, 2, 9, 4, 15, 6**.

3.1 Araya sokma sıralama (Insertion sort)

Mantık: Dizi iki parçaya ayrılır: solda sıralı kısım, sağda sırasız kısım. İlk eleman sıralı kabul edilir. Sırasız kısmın ilk elemanı elde tutulur (`ekle`), sıralı kısımla *sondan başa* karşılaştırılır; kendisinden büyük olanlar bir sağa kayarak *yer açar*, boşluğa `ekle` yerleştirilir (iskambil kağıdı dizme gibi).

```
void arayaSokma(int D[], int n) {
  int i, k, ekle;                          // ihmal
  for (i = 1; i < n; i++) {                // 1. eleman yerinde kalir
     ekle = D[i];                          // siradaki eleman elde
     for (k = i-1; k >= 0 && ekle <= D[k]; k--)
        D[k+1] = D[k];                     // buyuk eleman bir saga kayar
     D[k+1] = ekle;                        // bosluga yerlestir
  }
}
```

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

i	ekle	kayan eleman	Dizi
–	–	–	7, 3, 5, 8, 2, 9, 4, 15, 6 (ilk eleman sıralı kabul edilir)
1	3	1	**3**, 7, 5, 8, 2, 9, 4, 15, 6
2	5	1	3, **5**, 7, 8, 2, 9, 4, 15, 6
3	8	0	3, 5, 7, **8**, 2, 9, 4, 15, 6
4	2	4	**2**, 3, 5, 7, 8, 9, 4, 15, 6
5	9	0	2, 3, 5, 7, 8, **9**, 4, 15, 6
6	4	4	2, 3, **4**, 5, 7, 8, 9, 15, 6
7	15	0	2, 3, 4, 5, 7, 8, 9, **15**, 6
8	6	4	2, 3, 4, 5, **6**, 7, 8, 9, 15

Sonuç: 2, 3, 4, 5, 6, 7, 8, 9, 15

- **En kötü durum:** dizi büyükten küçüğe sıralı – her eleman en başa kadar kayar. **Tüm elemanlar eşitse de en kötü durumdur** çünkü koşul `ekle <= D[k]` eşitlikte de döngüye girer [video 5 37:43].
- **En iyi durum:** dizi küçükten büyüğe sıralı – kaydırma satırı hiç çalışmaz.
- Dış for: $1+n+(n-1)=2n$; `ekle` ataması $n-1$; iç döngü toplam $1+2+\cdots+(n-1)=\frac{n^2-n}{2}$ $\Rightarrow T(n)$ ikinci dereceden: $O(n^2)$, en iyi $\Omega(n)$.

Satır satır yürütme zamanı (değişken tanımları ihmal edilmiştir)

Satır	En iyi	En kötü
`for (i = 1; i < n; i++)`	$2n$	$2n$
`ekle = D[i]`	$n-1$	$n-1$
`for (k = i-1; k >= 0 && ekle <= D[k]; k--)`	$2(n-1)$	n^2+n-2
`D[k+1] = D[k]` (kaydırma)	0	$n(n-1)/2$
`D[k+1] = ekle`	$n-1$	$n-1$
$T(n)$	$\mathbf{6n-4}$	$\mathbf{(3n^2+9n-8)/2}$

Zaman ve alan karmaşıklığı

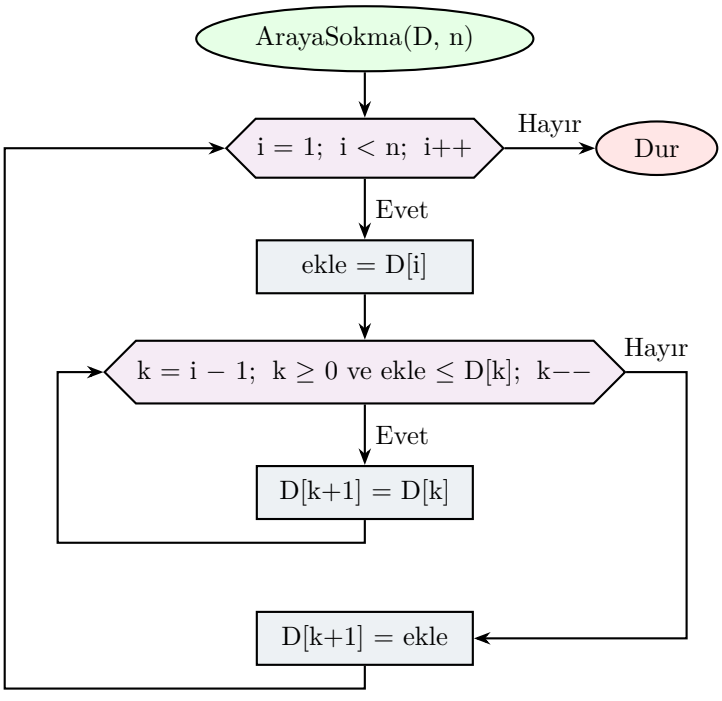
Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(n)$	dizi küçükten büyüğe sıralı: iç döngü koşulu ilk kontrolde yanlış, kaydırma yok
En kötü	$O(n^2)$	dizi büyükten küçüğe sıralı *veya tüm elemanlar eşit* (`<=` yüzünden eşitte de kayar)
Ortalama	–	en iyi n, en kötü n^2 dereceli $\Rightarrow \Theta$ verilmez
Ek alan	$O(1)$	sadece `ekle` değişkeni; dizi yerinde sıralanır

Araya sokma – nihai özet

Özellik	En iyi durum	En kötü durum
Eleman karşılaştırması (`ekle <= D[k]`)	$n-1$	$n(n-1)/2$
Kaydırma (`D[k+1] = D[k]`)	0	$n(n-1)/2$
$T(n)$	$6n-4$	$(3n^2+9n-8)/2$
Zaman karmaşıklığı	$\Omega(n)$	$O(n^2)$
Ek alan	$O(1)$	$O(1)$

Akış diyagramı: araya sokma sıralama

İç döngü de kodda `for` olduğu için altıgen döngü sembolüyle çizilmiştir: başlangıç `k = i-1`, koşul `k >= 0 && ekle <= D[k]`, artım `k--`; gövdesi yalnızca kaydırma satırıdır, döngü bitince (Hayır) `D[k+1] = ekle` yapılır.



3.2 Seçmeli sıralama (Selection sort)

Mantık: Dizideki en küçük eleman *seçilir* ve en başa konur; kalanlar içinden en küçük seçilip ikinci sıraya konur…

```
void secmeli(int D[], int n) {
  int i, j, enk, indis;                    // ihmal
  for (i = 0; i < n-1; i++) {
     enk = D[n-1];  indis = n-1;           // EN SONDAKI eleman en kucuk kabul edilir
     for (j = i; j < n-1; j++)
        if (D[j] < enk) { enk = D[j]; indis = j; }
     D[indis] = D[i];                      // bastaki eleman en kucugun yerine
     D[i] = enk;                           // en kucuk basa
  }
}
```

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

i	Seçilen en küçük	Dizi (yer değiştirme sonrası)
0	2	**2**, 3, 5, 8, **7**, 9, 4, 15, 6
1	3	2, **3**, 5, 8, 7, 9, 4, 15, 6
2	4	2, 3, **4**, 8, 7, 9, **5**, 15, 6
3	5	2, 3, 4, **5**, 7, 9, **8**, 15, 6
4	6	2, 3, 4, 5, **6**, 9, 8, 15, **7**
5	7	2, 3, 4, 5, 6, **7**, 8, 15, **9**
6	8	2, 3, 4, 5, 6, 7, **8**, 15, 9
7	9	2, 3, 4, 5, 6, 7, 8, **9**, **15**

Sonuç: 2, 3, 4, 5, 6, 7, 8, 9, 15

! Hoca derste vurguladı [video 5 Sıralama 2 53:01–59:12]

Hocanın kodunda başlangıç en küçüğü *en sondaki* elemandır. Bu yüzden:
En iyi durum: dizi *tersten (büyükten küçüğe)* sıralı veya *tüm elemanlar eşit* – if içi hiç çalışmaz. (Küçükten büyüğe sıralı dizi en iyi durum *değildir*: son eleman en büyük olduğundan ilk karşılaştırmada if içine girilir.)
En kötü durum: son eleman en büyük ve geri kalanı büyükten küçüğe sıralı – if içine her seferinde girilir.
"Verilen koda göre düşünmemiz lazım… En küçüğü en baştakini alırsam senaryoyu ona göre ayarlamam lazım."

İç döngü toplam $(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}$; en iyi ve en kötü durum ikisi de n^2 dereceli $\Rightarrow \Theta(n^2)$.

Satır satır yürütme zamanı (değişken tanımları ihmal edilmiştir)

Satır	En iyi	En kötü
`for (i = 0; i < n-1; i++)`	$2n$	$2n$
`enk = D[n-1]; indis = n-1`	$2(n-1)$	$2(n-1)$
`for (j = i; j < n-1; j++)`	n^2+n-2	n^2+n-2
`if (D[j] < enk)`	$n(n-1)/2$	$n(n-1)/2$
`enk = D[j]; indis = j`	0	$n(n-1)$
`D[indis] = D[i]; D[i] = enk`	$2(n-1)$	$2(n-1)$
$T(n)$	$\mathbf{(3n^2+13n-12)/2}$	$\mathbf{(5n^2+11n-12)/2}$

Zaman ve alan karmaşıklığı

Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(n^2)$	if içi hiç çalışmaz (tüm elemanlar eşit; hocaya göre tersten sıralı da)
En kötü	$O(n^2)$	if içi her seferinde çalışır (son eleman en büyük, gerisi azalan)
Ortalama	$\Theta(n^2)$	en iyi ve en kötü aynı derecede; hoca: *"ortalama durum maliyetinden bahsedebiliriz"*
Ek alan	$O(1)$	`enk` ve `indis`; dizi yerinde sıralanır

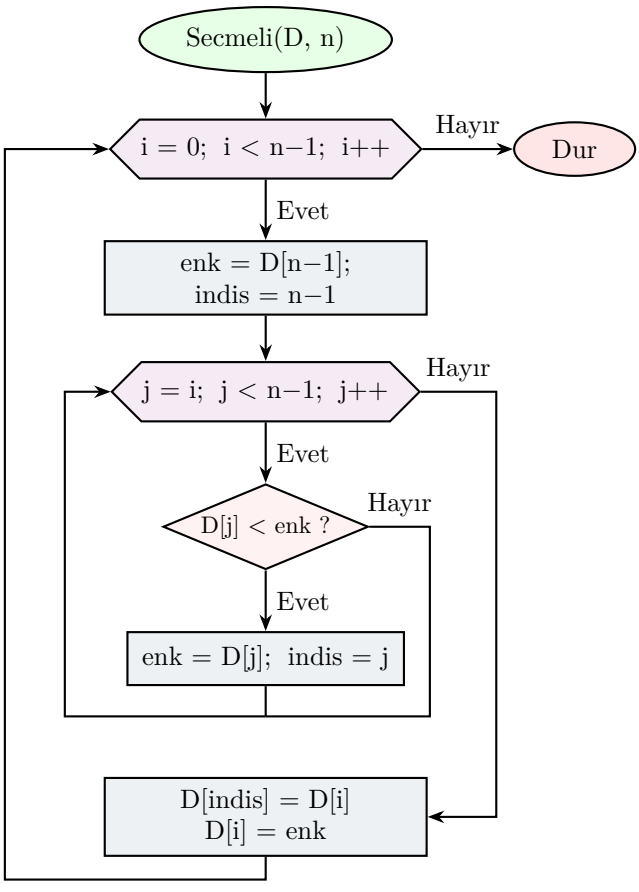
Seçmeli – nihai özet

Özellik	En iyi durum	En kötü durum
Eleman karşılaştırması (`D[j] < enk`)	$n(n-1)/2$	$n(n-1)/2$
if içi (yeni en küçük bulma)	0	$n(n-1)/2$ (üst sınır)
Yer değiştirme (her turda bir)	$n-1$	$n-1$
$T(n)$	$(3n^2+13n-12)/2$	$(5n^2+11n-12)/2$
Zaman karmaşıklığı	$\Omega(n^2)$	$O(n^2) \Rightarrow \Theta(n^2)$
Ek alan	$O(1)$	$O(1)$

Seçmelide en iyi/en kötü durumun inceliği

Tablodaki en iyi ve en kötü $T(n)$, hocanın analizindeki gibi *if içi hiç çalışmaz* ve *if içi her seferinde çalışır* kabulleriyle hesaplanmıştır. Simülasyonla kontrol edildiğinde: tüm elemanlar eşitse if içine gerçekten hiç girilmez; tersten sıralı dizide ise yalnız *ilk turda* hiç girilmez, ilk yer değiştirmeden sonra sondaki eleman değiştiği için sonraki turlarda kısmen girilir. Derece (n^2) her durumda aynı kaldığından sonuç değişmez: $\Theta(n^2)$. Sınavda hocanın kabulünü yazın.

Akış diyagramı: seçmeli sıralama (başlangıç en küçüğü en sondaki eleman)



3.3 Kabarcık sıralama (Bubble sort)

Mantık (deniz dalgası): Her "dalga hareketinde" komşu elemanlar karşılaştırılır, büyük olan sağa geçer. Bir hareket sonunda en büyük eleman en sona yerleşir, küçükler öne yaklaşır (çift yönlü etki). Her harekette iç döngü bir eksik çalışır.

```
void kabarcik(int D[], int n) {
  int gecici, k, hareket;                    // ihmal
  for (hareket = 0; hareket < n-1; hareket++)          // dalga hareketi
     for (k = 0; k < n-1-hareket; k++)                // komsu karsilastirma
        if (D[k] > D[k+1]) {
           gecici = D[k];  D[k] = D[k+1];  D[k+1] = gecici;
        }
}
```

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Hareket	k	İşlem	Dizi (işlem sonrası)
0	0	7 ↔ 3	**3**, **7**, 5, 8, 2, 9, 4, 15, 6
0	1	7 ↔ 5	3, **5**, **7**, 8, 2, 9, 4, 15, 6
0	2	değişim yok	3, 5, 7, 8, 2, 9, 4, 15, 6
0	3	8 ↔ 2	3, 5, 7, **2**, **8**, 9, 4, 15, 6
0	4	değişim yok	3, 5, 7, 2, 8, 9, 4, 15, 6
0	5	9 ↔ 4	3, 5, 7, 2, 8, **4**, **9**, 15, 6
0	6	değişim yok	3, 5, 7, 2, 8, 4, 9, 15, 6
0	7	15 ↔ 6	3, 5, 7, 2, 8, 4, 9, **6**, **15**
1	0	değişim yok	3, 5, 7, 2, 8, 4, 9, 6, 15
1	1	değişim yok	3, 5, 7, 2, 8, 4, 9, 6, 15
1	2	7 ↔ 2	3, 5, **2**, **7**, 8, 4, 9, 6, 15
1	3	değişim yok	3, 5, 2, 7, 8, 4, 9, 6, 15
1	4	8 ↔ 4	3, 5, 2, 7, **4**, **8**, 9, 6, 15
1	5	değişim yok	3, 5, 2, 7, 4, 8, 9, 6, 15
1	6	9 ↔ 6	3, 5, 2, 7, 4, 8, **6**, **9**, 15
2	0	değişim yok	3, 5, 2, 7, 4, 8, 6, 9, 15
2	1	5 ↔ 2	3, **2**, **5**, 7, 4, 8, 6, 9, 15
2	2	değişim yok	3, 2, 5, 7, 4, 8, 6, 9, 15
2	3	7 ↔ 4	3, 2, 5, **4**, **7**, 8, 6, 9, 15
2	4	değişim yok	3, 2, 5, 4, 7, 8, 6, 9, 15
2	5	8 ↔ 6	3, 2, 5, 4, 7, **6**, **8**, 9, 15
3	0	3 ↔ 2	**2**, **3**, 5, 4, 7, 6, 8, 9, 15
3	1	değişim yok	2, 3, 5, 4, 7, 6, 8, 9, 15
3	2	5 ↔ 4	2, 3, **4**, **5**, 7, 6, 8, 9, 15
3	3	değişim yok	2, 3, 4, 5, 7, 6, 8, 9, 15
3	4	7 ↔ 6	2, 3, 4, 5, **6**, **7**, 8, 9, 15
4	0	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
4	1	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
4	2	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
4	3	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
5	0	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
5	1	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
5	2	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
6	0	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
6	1	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
7	0	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15

Sonuç: 2, 3, 4, 5, 6, 7, 8, 9, 15

- En iyi durum: dizi küçükten büyüğe sıralı *veya tüm elemanlar eşit* (if içi hiç çalışmaz). En kötü: tersten sıralı (if içine her seferinde girilir).

- İç döngü toplamı $\frac{n(n-1)}{2}$; en iyi ve en kötü fark yaklaşık $3n^2/2$ terimi (yer değiştirmenin 3 ataması) $\Rightarrow \Omega(n^2)$, $O(n^2)$, $\Theta(n^2)$.

Satır satır yürütme zamanı (değişken tanımları ihmal edilmiştir)

Satır	En iyi	En kötü
`for (hareket = 0; hareket < n-1; hareket++)`	$2n$	$2n$
`for (k = 0; k < n-1-hareket; k++)`	n^2+n-2	n^2+n-2
`if (D[k] > D[k+1])`	$n(n-1)/2$	$n(n-1)/2$
yer değiştirme (3 atama)	0	$3n(n-1)/2$
$T(n)$	$\mathbf{(3n^2+5n-4)/2}$	$\mathbf{3n^2+n-2}$

Zaman ve alan karmaşıklığı

Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(n^2)$	dizi küçükten büyüğe sıralı veya tüm elemanlar eşit: if içi hiç çalışmaz ama döngüler yine tam döner
En kötü	$O(n^2)$	dizi büyükten küçüğe sıralı: her karşılaştırmada yer değiştirme
Ortalama	$\Theta(n^2)$	en iyi ve en kötü aynı derecede
Ek alan	$O(1)$	sadece `gecici` değişkeni; dizi yerinde sıralanır

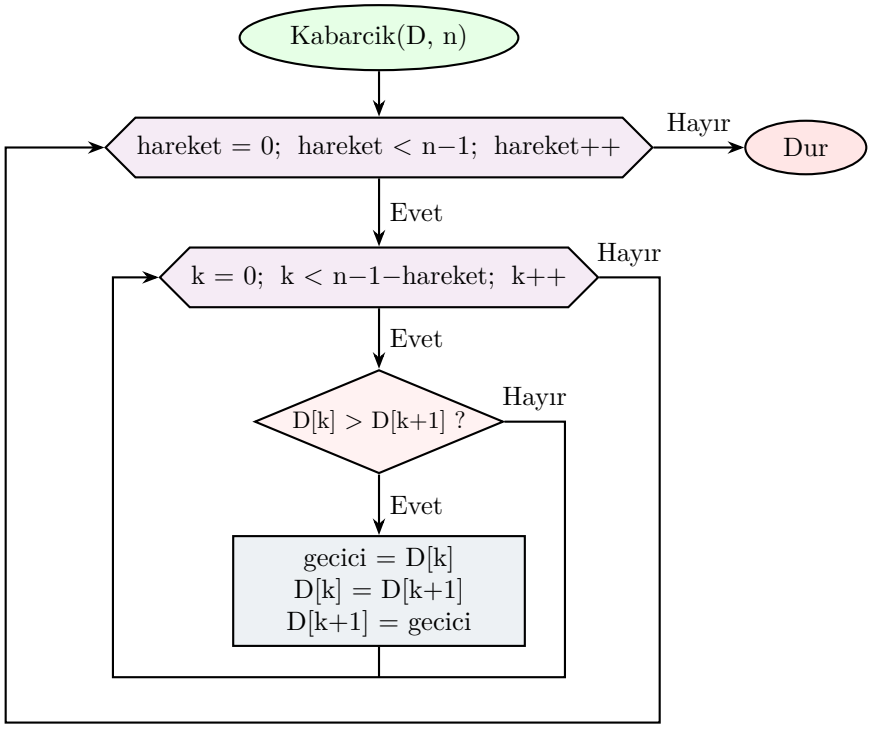
Kabarcık – nihai özet

Özellik	En iyi durum	En kötü durum
Eleman karşılaştırması (`D[k] > D[k+1]`)	$n(n-1)/2$	$n(n-1)/2$
Yer değiştirme	0	$n(n-1)/2$
$T(n)$	$(3n^2+5n-4)/2$	$3n^2+n-2$
Zaman karmaşıklığı	$\Omega(n^2)$	$O(n^2) \Rightarrow \Theta(n^2)$
Ek alan	$O(1)$	$O(1)$

Başka kaynaklarda "en iyi durum $O(n)$" görürseniz

O kaynaklar, bir dalgada hiç yer değiştirme olmazsa döngüden çıkan bir *bayrak* ekler. Hocanın kodunda bu bayrak yoktur; sıralı dizide de bütün dalgalar döner, bu yüzden en iyi durum $\Omega(n^2)$'dir. Bir öğrencinin WhatsApp'ta paylaştığı satır satır tablo da aynı sonuca varır (en iyi ve en kötü ikisi de n^2 dereceli).

Akış diyagramı: kabarcık sıralama



3.4 Birleşmeli sıralama (Merge sort)

Mantık (böl ve fethet): Dizi **pivot = (ilk indis + son indis)/2, aşağı yuvarlanır** ile ikiye bölünür (pivot ve solu sol parçada). Tek eleman kalana kadar bölünür. Sonra sıralı parçalar *ilk elemanları karşılaştırılarak* birleştirilir.

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Bölme aşaması (pivot = (ilk+son)/2 aşağı yuvarlanır, pivot ve solu sol parçaya):

[7, 3, 5, 8, 2] | [9, 4, 15, 6]

· [7, 3, 5] | [8, 2]

· · [7, 3] | [5]

· · · [7] | [3]

· · [8] | [2]

· [9, 4] | [15, 6]

· · [9] | [4]

· · [15] | [6]

Birleştirme aşaması (sıralı parçaların ilk elemanları karşılaştırılır):

[7] + [3] ⇒ [3, 7]

[3, 7] + [5] ⇒ [3, 5, 7]

[8] + [2] ⇒ [2, 8]

[3, 5, 7] + [2, 8] ⇒ [2, 3, 5, 7, 8]

[9] + [4] ⇒ [4, 9]

[15] + [6] ⇒ [6, 15]

$[4, 9] + [6, 15] \Rightarrow [4, 6, 9, 15]$

$[2, 3, 5, 7, 8] + [4, 6, 9, 15] \Rightarrow [2, 3, 4, 5, 6, 7, 8, 9, 15]$

Sonuç: 2, 3, 4, 5, 6, 7, 8, 9, 15

- Bölme $h = \log_2 n$ seviye; her seviyede n eleman üzerinden geçilir $\Rightarrow O(n \log n)$ (en iyi ve en kötü aynı: $\Theta(n \log n)$).
- **En büyük dezavantajı alan karmaşıklığı**: özyinelemeli çalıştığı için her bölmede fonksiyon kendi kopyasını oluşturur, bellek hızlı dolar [video 6 01:05:51].

```
Birlesmeli(D, ilk, son)
  if (ilk < son)
     p = (ilk + son) / 2          // asagi yuvarlanir
     Birlesmeli(D, ilk, p)
     Birlesmeli(D, p+1, son)
     Birlestir(D, ilk, p, son)

Birlestir(D, ilk, p, son)
  i = ilk;  j = p+1;  t = 0
  while (i <= p && j <= son)
     if (D[i] <= D[j]) G[t++] = D[i++]
     else              G[t++] = D[j++]
  while (i <= p)   G[t++] = D[i++]
  while (j <= son) G[t++] = D[j++]
  G'yi D[ilk..son]'a kopyala
```

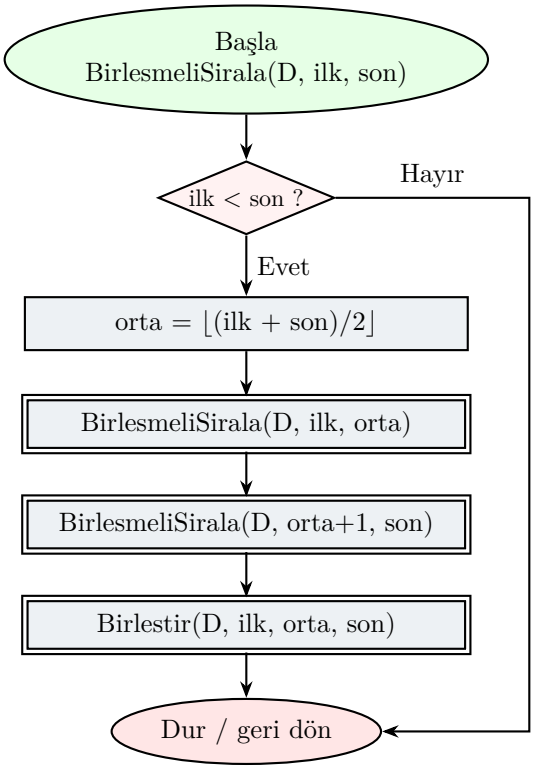
Zaman ve alan karmaşıklığı

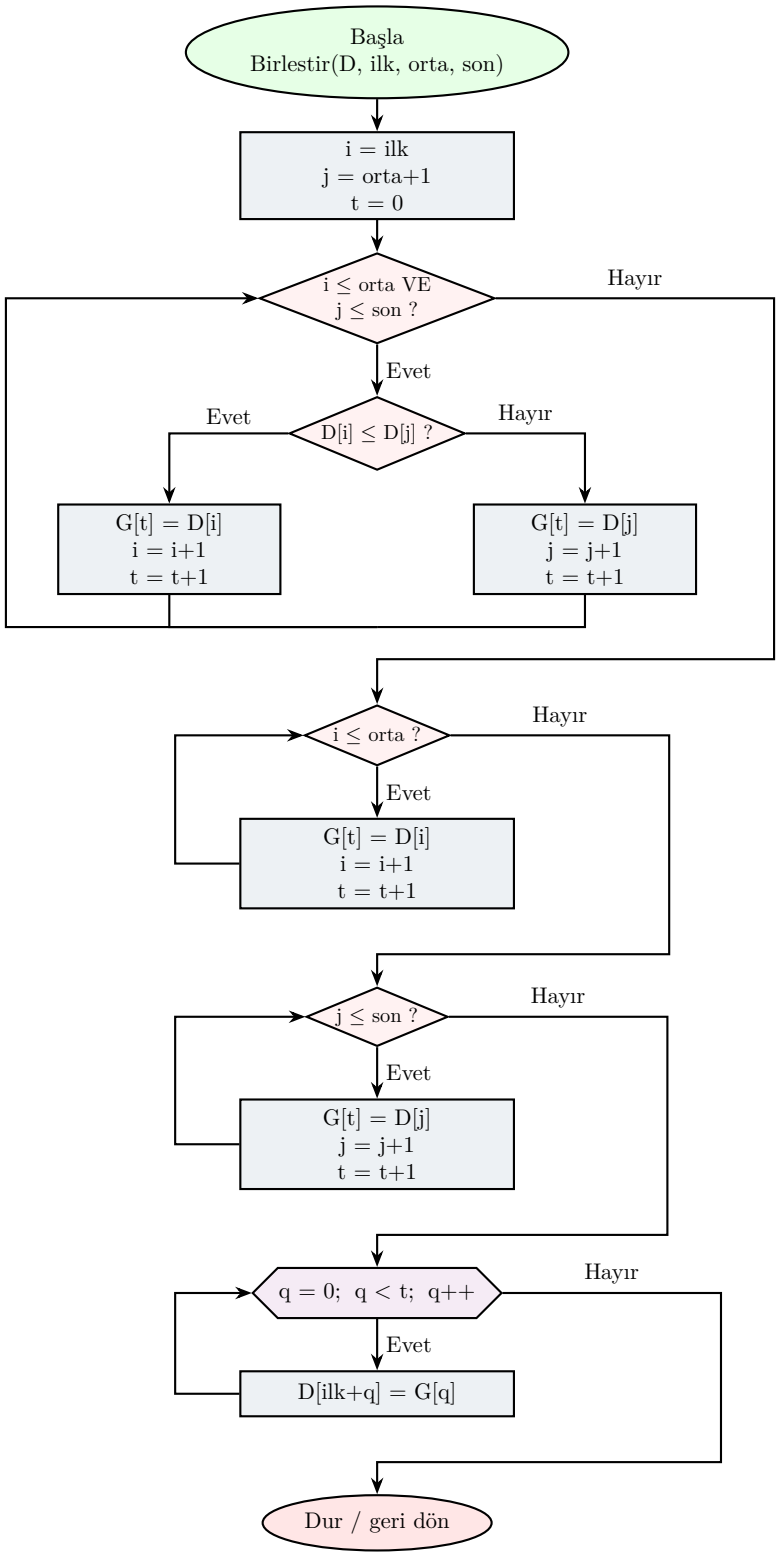
Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(n \log n)$	bölme her zaman $\log_2 n$ seviye; her seviyede n eleman birleştirilir
En kötü	$O(n \log n)$	verinin geliş sırası bölmeyi değiştirmez
Ortalama	$\Theta(n \log n)$	en iyi ve en kötü aynı derecede
Ek alan	$O(n)$	geçici dizi `G` + özyinelemeli çağrıların kopyaları; hocanın vurguladığı dezavantaj

Birleşmeli – nihai özet

Özellik	En iyi durum	En kötü durum
Eleman karşılaştırması	$\approx n/2 \log_2 n$ (sıralı dizi)	$\approx n \log_2 n$
Kopyalama (`G`'ye ve geri)	$2n \log_2 n$	$2n \log_2 n$
$T(n)$	$T(n) = 2T(n/2) + cn \Rightarrow \approx cn \log_2 n$	
Zaman karmaşıklığı	$\Omega(n \log n)$	$O(n \log n) \Rightarrow \Theta(n \log n)$
Ek alan	$O(n)$	$O(n)$

Akış diyagramı: birleşmeli sıralama (ana fonksiyon ve birleştirme)





3.5 Kümeleme sıralama (Heap sort)

! Hoca derste vurguladı [video 6 Sıralama 3 44:25, 57:40]

İki aşaması vardır: A) hip ağacının oluşturulması, B) sıralama (koparma). *"Soru sorulduğu zaman bir kısmınız ağacı oluşturuyor ve bırakıyor. Sıralama işlemini gerçekleştirmiyor. O doğru cevap değil. Cevap eksik."* Ayrıca *"İkili arama ağacıyla karıştırmayın"* – hip ağacında küçükler sola, büyükler sağa kuralı yoktur.

Hip özelliği (ders varsayımı): her düğüm çocuklarına eşit veya onlardan *büyüktür* (max-heap); tersi de olabilir ama bir kural seçilip sonuna kadar uygulanır.

1. **Ekleme:** Zigzak mantığı – seviye seviye, soldan sağa (kök, sonra 2. kademe soldan sağa, sonra 3. kademe…). Her eklemeden sonra eklenen eleman ebeveyninden büyükse yer değiştirerek yukarı çıkar. *"Hip özelliği sağlanmıyorsa veri eklemeye devam edemeyiz... sonuç doğru çıksa da algoritma o şekilde çalışmaz."*
2. **Koparma:** Kök (en büyük) koparılır ve *kenara yazılır* (*"neyi nasıl sıraladığınızı göstermeniz gerekiyor"*). Zigzak sırasına göre en son eleman köke gelir ve *büyük olan çocukla* yer değiştirerek aşağı iner. Ağaç boşalana kadar devam edilir.

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

A) Hip (kümeleme) ağacının oluşturulması — zigzag (seviye seviye, soldan sağa) ekleme; her eklemede düğüm ≥ çocukları kontrolü:

7 eklendi; hip sağlanıyor



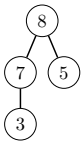
3 eklendi; hip sağlanıyor



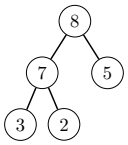
5 eklendi; hip sağlanıyor



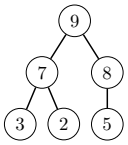
8 eklendi; 8 ↔ 3, 8 ↔ 7



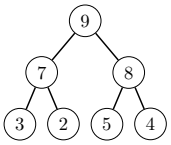
2 eklendi; hip sağlanıyor



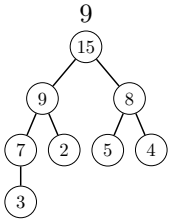
9 eklendi; 9 ↔ 5, 9 ↔ 8



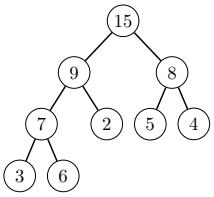
4 eklendi; hip sağlanıyor



15 eklendi; 15 ↔ 3, 15 ↔ 7, 15 ↔ 9

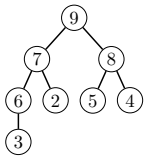


6 eklendi; hip sağlanıyor

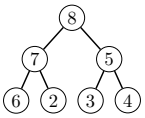


B) Sıralama (koparma) aşaması — kök koparılıp kenara yazılır, zigzag sırasına göre en son eleman köke gelir ve *büyük olan çocukla* yer değiştirerek aşağı iner:

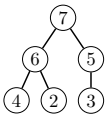
15 koparıldı, 6 köke geldi; 6 ↔ 9, 6 ↔ 7
Kopanlar: 15



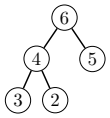
9 koparıldı, 3 köke geldi; 3 ↔ 8, 3 ↔ 5
Kopanlar: 15, 9



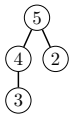
8 koparıldı, 4 köke geldi; 4 ↔ 7, 4 ↔ 6
Kopanlar: 15, 9, 8



7 koparıldı, 3 köke geldi; 3 ↔ 6, 3 ↔ 4
Kopanlar: 15, 9, 8, 7



6 koparıldı, 2 köke geldi; 2 ↔ 5
Kopanlar: 15, 9, 8, 7, 6



5 koparıldı, 3 köke geldi; 3 ↔ 4
Kopanlar: 15, 9, 8, 7, 6, 5



4 koparıldı, 2 köke geldi; 2 ↔ 3
Kopanlar: 15, 9, 8, 7, 6, 5, 4



3 koparıldı, 2 köke geldi
Kopanlar: 15, 9, 8, 7, 6, 5, 4, 3

2 koparıldı
Kopanlar: 15, 9, 8, 7, 6, 5, 4, 3, 2

Koparılanlar büyükten küçüğe çıktığı için sondan başa yazılır. **Sıralı dizi:** 2, 3, 4, 5, 6, 7, 8, 9, 15

Çıkmış sorular: 2020 Güz Vize 1a (sıralama), **2020 Güz Bütünleme 1a–c** (sıralama + akış diyagramı + yürütme zamanı) ve 2021 Yaz Final 1a–b (sıralama + akış diyagramı). Vize ve bütünleme çözümleri: `02_Vize_Cozumleri.pdf`. **Karmaşıklık:** ağaç yüksekliği $\log n$; her eleman için ekleme/koparma $O(\log n) \Rightarrow O(n \log n)$.

Kod (hocanın slayt stilinde, iki yardımcı fonksiyonla): Hoca sınavda genelde *"ya kaba kodu ya da akış diyagramı"* istiyor; kod, bölüm sonundaki akış diyagramlarıyla (`Kumeleme`, `HipEkle`, `KokSil`) birebir aynıdır.

```
Kumeleme ( int D[], int N)
{
    int H[N];                    // hip agaci, H[0] = kok
    int i, k;

    for (i=0; i<N; i++)          // A) AGAC OLUSTURMA
        HipEkle(H, i, D[i]);     // teker teker ekle

    for (k=N-1; k>=0; k--) {     // B) SIRALAMA
        D[k] = H[0];             // koku kopar, kenara yaz
        KokSil(H, k);            // sondakini koke koy
    }
}

HipEkle ( int H[], int son, int x)
{                                // x'i zigzag'a gore ekle
    int c, ebeveyn, gecici;
    H[son] = x;                  // ilk bos yer
    c = son;
    ebeveyn = (c-1)/2;
    while (c>0 && H[c]>H[ebeveyn]) {
        gecici = H[c];           // yukari tasi
        H[c] = H[ebeveyn];
        H[ebeveyn] = gecici;
        c = ebeveyn;
        ebeveyn = (c-1)/2;
    }
}

KokSil ( int H[], int son)
{                                // agac: H[0..son]
    int c, buyuk, gecici;
    H[0] = H[son];               // en sondaki koke
    son = son-1;
    c = 0;
    while (2*c+1 <= son) {       // cocugu var
        buyuk = 2*c+1;           // sol cocuk
        if (2*c+2 <= son && H[2*c+2] > H[buyuk])
            buyuk = 2*c+2;       // sag daha buyuk
        if (H[c] >= H[buyuk])    // hip saglandi
            break;
        gecici = H[c];           // BUYUK cocukla
        H[c] = H[buyuk];         // yer degistir
        H[buyuk] = gecici;
        c = buyuk;
    }
}
```

! Hoca derste vurguladı [video 2 Akış Diyagramı 58:33]

Kendi yazdığın fonksiyonu çağırıyorsan *"o fonksiyonun detaylarını veya akış diyagramını vermemiz gerekecek."* Yani `HipEkle` ve `KokSil`'in içi de yazılmalı; yalnızca ana fonksiyonu yazmak eksik cevaptır. Kelime sorusunda `>` yerine "alfabede sonra gelir" karşılaştırması kullanılır (birleşik alfabe: … P Q R S Ş T U Ü V W X Y Z).

Sık hatalar: küçük çocukla yer değiştirmek; eklerken `>=` kullanmak (eşitler gereksiz yer değiştirir); 0'dan başlayan indislerde ebeveyni `c/2` almak (doğrusu `(c-1)/2`); ders kitabındaki `BuildMaxHeap` (`for i = n/2 downto 1`) yöntemini yazmak – o yöntem ağacı hocanınkinden farklı kurar. Hip ağacında indisler (zigzag sırası): ebeveyn $(c-1)/2$, çocuklar $2c+1$ ve $2c+2$.

Zaman ve alan karmaşıklığı

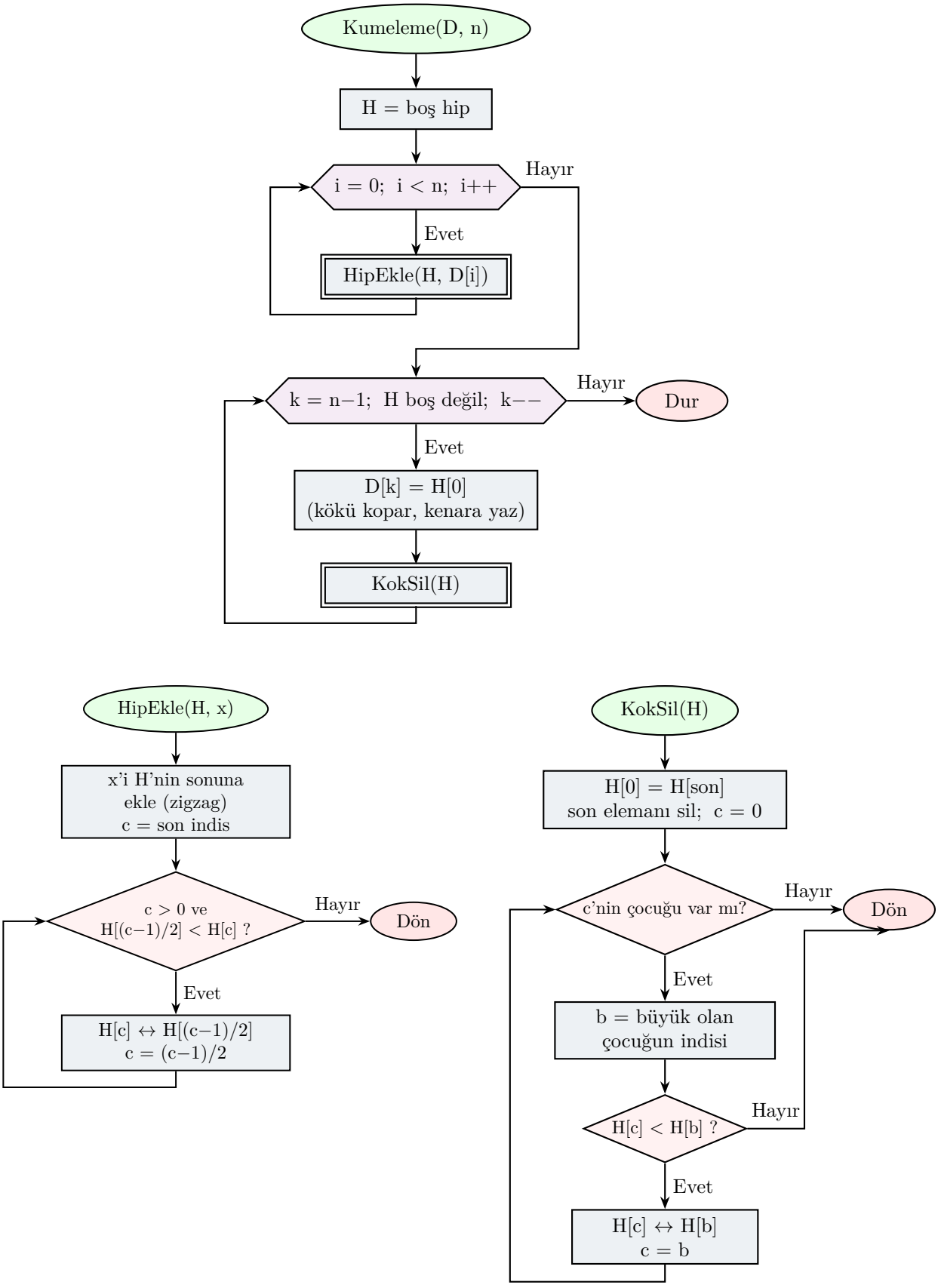
Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(n \log n)$	her eleman için yukarı taşıma/aşağı indirme en fazla ağaç yüksekliği ($\log_2 n$) kadar
En kötü	$O(n \log n)$	n ekleme + n koparma, her biri $O(\log n)$
Ortalama	$\Theta(n \log n)$	en iyi ve en kötü aynı derecede
Ek alan	$O(n)$	hocanın yönteminde hip ayrı bir dizide (`H`) tutulur

Kümeleme – nihai özet

Özellik	En iyi durum	En kötü durum
A) Ağaç oluşturma (`HipEkle`)	n ekleme	n ekleme $\times \log_2 n$ yukarı taşıma
B) Koparma (`KokSil`)	n koparma	n koparma $\times \log_2 n$ aşağı indirme
$T(n)$		$\approx c_1 n \log_2 n + c_2 n \log_2 n$
Zaman karmaşıklığı	$\Omega(n \log n)$	$O(n \log n) \Rightarrow \Theta(n \log n)$
Ek alan	$O(n)$	$O(n)$

Akış diyagramı: kümeleme sıralama (iki aşama + iki yardımcı fonksiyon)

"Ağacı oluşturup bırakmak eksik cevaptır": akış diyagramında da *iki döngü* (oluşturma ve koparma) görünmelidir.



3.6 Hızlı sıralama (Quick sort)

Mantık: Bir *pivot* seçilir; pivottan küçükler sola, büyükler sağa geçer ve pivot *sıralamadaki yerini bulur*. Sonra sol ve sağ parçalara aynı işlem uygulanır. Birleşmeliden farkı: *"birleştirme adımı yok, bölerken sıralıyor"*.

! Hoca derste vurguladı [video 7 Sıralama 4 17:08, 35:41]

"Pivotu belirleme serbestliği hızlı sıralamada bize ait" (en sondaki, en baştaki, (ilk+son)/2, ortanca…). Ama *"bir sayıya odaklanmamak lazım"*; **genel geçer bir kural belirleyip bütün adımlarda uygulamamız lazım**. Sınav sorusunda kural verilir: 2020 "en sondaki kelimeyi pivot seçiniz", 2024 "en baştaki kelimeyi pivot olarak seçiniz". Eşit elemanlar pivotun yanına (genelde küçükler grubuna) alınır.

Tek dizi üzerinde yer değiştirme (hocanın yöntemi, pivot en sonda): soldan her eleman pivotla karşılaştırılır. Küçükse yerinde kalır. Büyükse 3 adım: (1) eleman pivotun yerine yazılır, (2) pivotun hemen önündeki eleman boşalan yere yazılır, (3) pivot bir sola kayar. Yeni gelen eleman tekrar kontrol edilir. Pivot ortadaysa sağındaki küçükler aynı mantıkla soluna alınır.

```
HizliSirala(D, ilk, son)
   if (ilk < son)
      p = Bolumle(D, ilk, son)
      HizliSirala(D, ilk, p-1)
      HizliSirala(D, p+1, son)

Bolumle(D, ilk, son)                  // pivot: en sondaki
   pivot = D[son];  p = son;  i = ilk
   while (i < p)
      if (D[i] > pivot)
         D[p] = D[i];  D[i] = D[p-1];  D[p-1] = pivot;  p = p-1
      else  i = i+1
   return p
```

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Pivot kuralı: dizinin (alt dizinin) EN SONUNDAKİ eleman pivot seçilir; bu kural *her adımda* aynen uygulanır. Pivottan büyükler sağa, küçükler sola geçer. Yer değiştirme hocanın tek dizi yöntemiyle yapılır: eleman pivotun yerine yazılır, pivotun yanındaki eleman boşalan yere yazılır, pivot bir birim kayar.

Başlangıç: 7, 3, 5, 8, 2, 9, 4, 15, 6

Alt dizi [0..8], pivot = **6**

7 > 6 olduğundan pivotun sağına: 15, 3, 5, 8, 2, 9, 4, **6**, 7

15 > 6 olduğundan pivotun sağına: 4, 3, 5, 8, 2, 9, **6**, 15, 7

8 > 6 olduğundan pivotun sağına: 4, 3, 5, 9, 2, **6**, 8, 15, 7

9 > 6 olduğundan pivotun sağına: 4, 3, 5, 2, **6**, 9, 8, 15, 7

Sonuç: [4, 3, 5, 2] **6** [9, 8, 15, 7] ⇒ 6 sıralamadaki yerini buldu (indis 4).

Alt dizi [0..3], pivot = **2**

4 > 2 olduğundan pivotun sağına: 5, 3, **2**, 4

5 > 2 olduğundan pivotun sağına: 3, **2**, 5, 4

3 > 2 olduğundan pivotun sağına: **2**, 3, 5, 4

Sonuç: [] **2** [3, 5, 4] ⇒ 2 sıralamadaki yerini buldu (indis 0).

Alt dizi [1..3], pivot = **4**

5 > 4 olduğundan pivotun sağına: 3, **4**, 5

Sonuç: [3] **4** [5] ⇒ 4 sıralamadaki yerini buldu (indis 2).

▷ [1..1] alt dizisinde tek eleman (3) kaldı ⇒ yerini buldu.

▷ [3..3] alt dizisinde tek eleman (5) kaldı ⇒ yerini buldu.

Alt dizi [5..8], pivot = **7**

9 > 7 olduğundan pivotun sağına: 15, 8, **7**, 9

15 > 7 olduğundan pivotun sağına: 8, **7**, 15, 9

8 > 7 olduğundan pivotun sağına: **7**, 8, 15, 9

Sonuç: [] **7** [8, 15, 9] ⇒ 7 sıralamadaki yerini buldu (indis 5).

Alt dizi [6..8], pivot = **9**

15 > 9 olduğundan pivotun sağına: 8, **9**, 15

Sonuç: [8] **9** [15] ⇒ 9 sıralamadaki yerini buldu (indis 7).

▷ [6..6] alt dizisinde tek eleman (8) kaldı ⇒ yerini buldu.

▷ [8..8] alt dizisinde tek eleman (15) kaldı ⇒ yerini buldu.

Sıralı dizi: 2, 3, 4, 5, 6, 7, 8, 9, 15

- **En kötü durum:** pivot hep en büyük ya da en küçük eleman (ör. sıralı dizi + son eleman pivot). Bölme olmaz, her seferinde $n-1$ eleman kalır ⇒ $T(n) = T(n-1) + cn$ ⇒ $O(n^2)$. *"Literatürdeki kaynaklara bakarsak da o şekildedir."*
- **En iyi durum:** pivot diziyi ortadan ikiye böler ⇒ $\log n$ seviye × n ⇒ $\Omega(n \log n)$.
- İyi pivot için öneriler: ortanca (ama bulmak için sıralama gerekir), ortalamaya en yakın değer, frekans/istatistik.
- Dezavantajlar: uygun pivot seçme zorluğu ve özyinelemeden dolayı yüksek alan maliyeti.
- Satır satır yürütme zamanının tam çözümü: `02_Vize_Cozumleri.pdf`, 2024–25 Vize Soru 2b.

Zaman ve alan karmaşıklığı

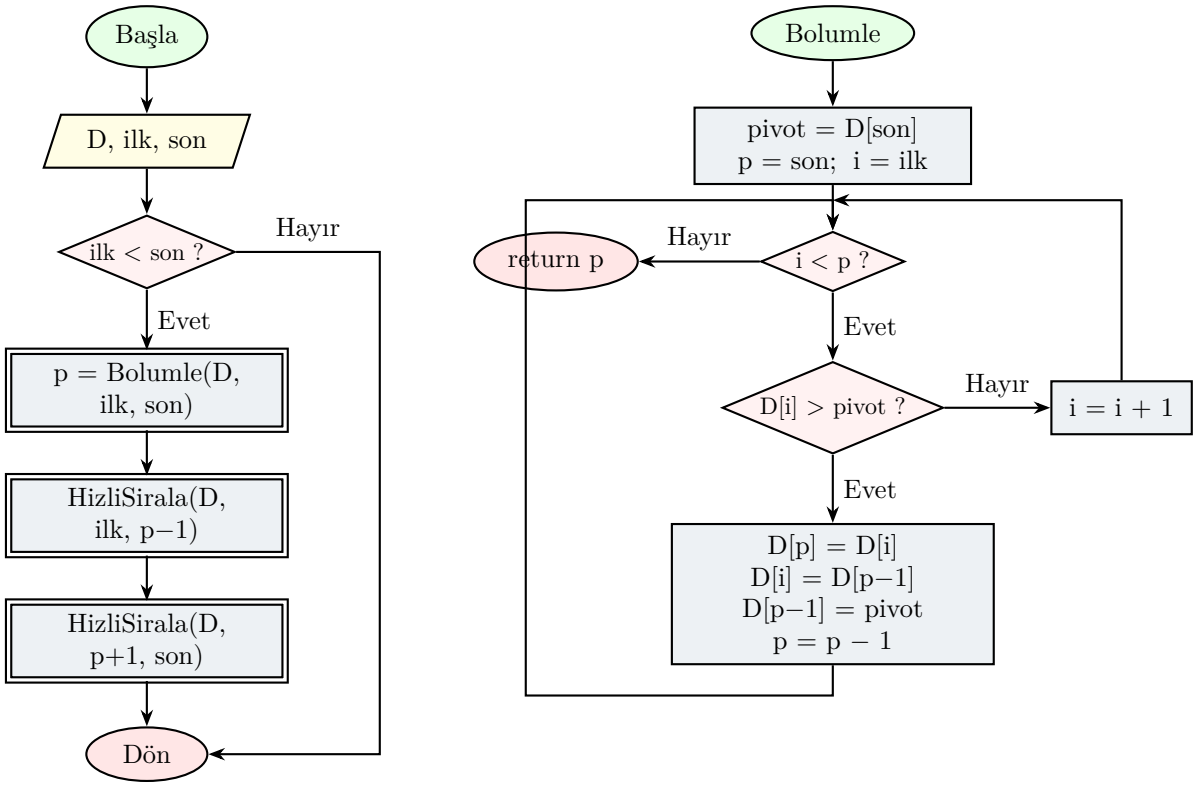
Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(n \log n)$	pivot her seferinde diziyi ortadan ikiye böler: $\log_2 n$ seviye × n
En kötü	$O(n^2)$	pivot hep en büyük/en küçük (ör. sıralı dizi + sondaki pivot): her seferinde $n-1$ eleman kalır
Ortalama	–	en iyi $n \log n$, en kötü n^2 dereceli ⇒ Θ verilmez
Ek alan	$O(\log n)$ – $O(n)$	dizi yerinde bölünür ama özyineleme derinliği kadar fonksiyon kopyası (en iyi $\log n$, en kötü n)

Hızlı – nihai özet

Özellik	En iyi durum	En kötü durum
Bölümleme $T_B(m)$ (m eleman)	$3m+2$	$6m-1$
Eleman karşılaştırması (`D[i] > pivot`)	$\approx n \log_2 n$	$n(n-1)/2$
$T(n)$	$\approx 6n \log_2 n$	$\approx 3n^2 + 4n$
Zaman karmaşıklığı	$\Omega(n \log n)$	$O(n^2)$
Ek alan	$O(\log n)$	$O(n)$

Akış diyagramı: hızlı sıralama (ana fonksiyon ve bölümleme)

Son iki vizede üst üste soruldu. “Pivot en baştaki” denirse pivotun sağındakiler soldan sağa taranır, küçükler sola alınır.



3.7 Radiks sıralama (Radix sort)

Mantık: En önemsiz haneden (sayıda birler basamağı, kelimede *son harf*) başlanır; elemanlar o hanedeki değere göre *kovalara* (sayılarda 0–9 = 10 kova; Türkçe kelimelerde 29+ harf kadar kova) *geliş sırasıyla* dağıtılır, kovalar sırayla toplanır. Geçiş sayısı = en uzun elemanın hane sayısı.

- **Hane eşitleme:** Sayılarda değeri değiştirmemek için *başa 0* eklenir (87 → 087). Kelimelerde sözlük sırasını bozmamak için *sona* A eklenir (öğrenci aktarımı: *“radix için hoca kelimenin en sağına A ekleyip…” – “Sona”*; 2021 çözümü: QOAW, HTRA, LMİA, UAAA…). Kısa kelimeler aynı başlayan uzunlardan önce gelir.
- **Sayı örneği (ders):** 275, 087, 426, 061, 509, 170, 677, 503
  1. geçiş (birler): 170, 061, 503, 275, 426, 087, 677, 509
  2. geçiş (onlar): 503, 509, 426, 061, 170, 275, 677, 087
  3. geçiş (yüzler): **061, 087, 170, 275, 426, 503, 509, 677**

Kelime örneği (ders: Ahmet, Ali, Ayşe, Veli, Mehmet, Serkan):

Hane eşitleme (en uzun kelime 6 harf; sözlük sırası bozulmasın diye kelimelerin **sonuna** ‘A’ eklenir): AHMETA, ALİAAA, AYŞEAA, VELİAA, MEHMET, SERKAN

1. geçiş (sağdan 1. harfe göre kovalar): **A**: AHMETA, ALİAAA, AYŞEAA, VELİAA; **N**: SERKAN; **T**: MEHMET

⇒ AHMETA, ALİAAA, AYŞEAA, VELİAA, SERKAN, MEHMET

2. geçiş (sağdan 2. harfe göre kovalar): **A**: ALİAAA, AYŞEAA, VELİAA, SERKAN; **E**: MEHMET; **T**: AHMETA

⇒ ALİAAA, AYŞEAA, VELİAA, SERKAN, MEHMET, AHMETA

3. geçiş (sağdan 3. harfe göre kovalar): **A**: ALİAAA; **E**: AYŞEAA, AHMETA; **İ**: VELİAA; **K**: SERKAN; **M**: MEHMET

⇒ ALİAAA, AYŞEAA, AHMETA, VELİAA, SERKAN, MEHMET

4. geçiş (sağdan 4. harfe göre kovalar): **H**: MEHMET; **İ**: ALİAAA; **L**: VELİAA; **M**: AHMETA; **R**: SERKAN; **Ş**: AYŞEAA

⇒ MEHMET, ALİAAA, VELİAA, AHMETA, SERKAN, AYŞEAA

5. geçiş (sağdan 5. harfe göre kovalar): **E**: MEHMET, VELİAA, SERKAN; **H**: AHMETA; **L**: ALİAAA; **Y**: AYŞEAA

⇒ MEHMET, VELİAA, SERKAN, AHMETA, ALİAAA, AYŞEAA

6. geçiş (sağdan 6. harfe göre kovalar): **A**: AHMETA, ALİAAA, AYŞEAA; **M**: MEHMET; **S**: SERKAN; **V**: VELİAA

⇒ AHMETA, ALİAAA, AYŞEAA, MEHMET, SERKAN, VELİAA

Eklenen A'lar atılır. **Sonuç:** AHMET, ALİ, AYŞE, MEHMET, SERKAN, VELİ

Karmaşıklık: d hane, n eleman, k kova: $O(d \cdot (n+k))$; d sabit kabul edilirse doğrusal.

```
RadiksSirala(D, n)
   d = en uzun elemanin hane sayisi
   tum elemanlari d haneye esitle        // sayida basa 0, kelimede sona A
   for (h = d; h >= 1; h--)              // sagdan sola her hane
      kovalari bosalt
      for (i = 0; i < n; i++)
         kova[ D[i]'nin h. hanesi ].enqueue(D[i])
      j = 0
      for (her kova c, alfabe/rakam sirasiyla)
         while (!kova[c].isEmpty())  D[j++] = kova[c].dequeue()
   eklenen 0/A karakterlerini at
```

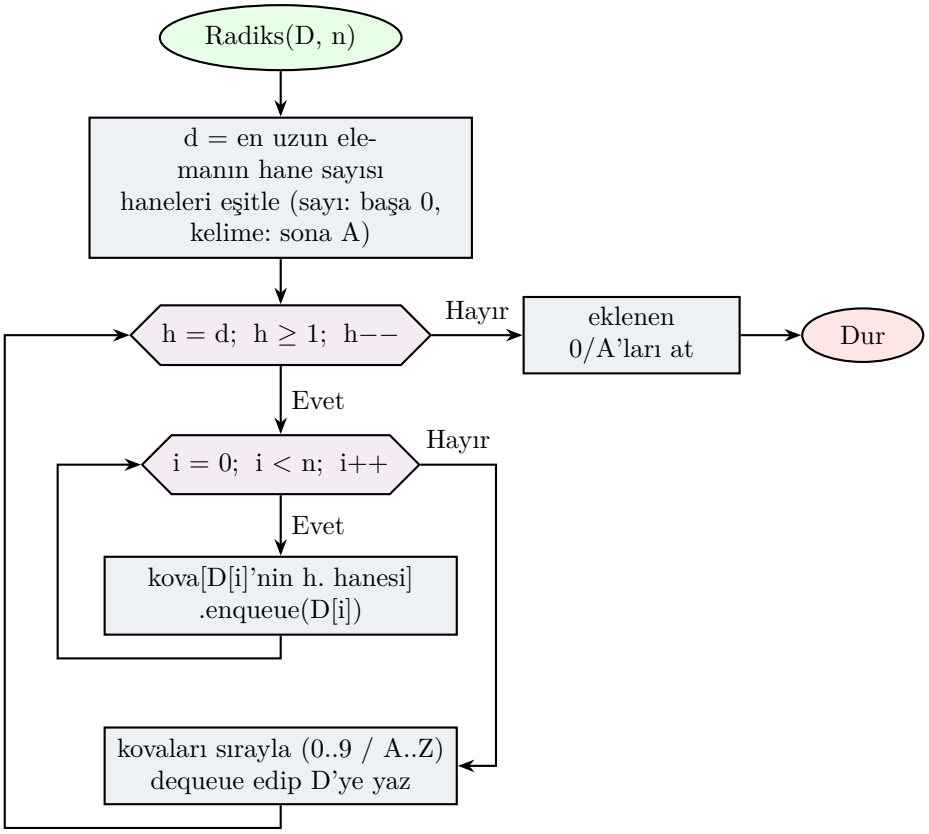
Zaman ve alan karmaşıklığı

Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(d(n+k))$	karşılaştırma yapılmaz; her geçişte n eleman dağıtılır, k kova toplanır
En kötü	$O(d(n+k))$	verinin sırası iş miktarını değiştirmez; d = en uzun elemanın hane sayısı
Ortalama	$\Theta(d(n+k))$	en iyi ve en kötü aynı; d sabitse doğrusal
Ek alan	$O(n+k)$	kovalar (kuyruklar) bütün elemanları geçici olarak tutar

Radiks – nihai özet

Özellik	En iyi durum	En kötü durum
Eleman karşılaştırması	0	0
Kovaya dağıtma (`enqueue`)	$d \cdot n$	$d \cdot n$
Kovadan toplama (`dequeue`)	$d \cdot n$	$d \cdot n$
$T(n)$	$\approx c\,d\,(n+k)$	
Zaman karmaşıklığı	$\Theta(d(n+k))$	
Ek alan	$O(n+k)$	$O(n+k)$

Akış diyagramı: radiks sıralama



▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

3.8 Özet tablo

Algoritma	En iyi	En kötü	Θ	Ek alan	Not
Araya sokma	$\Omega(n)$	$O(n^2)$	–	$O(1)$	en iyi: sıralı; en kötü: ters sıralı veya hepsi eşit
Seçmeli	$\Omega(n^2)$	$O(n^2)$	$\Theta(n^2)$	$O(1)$	hoca kodu: en iyi ters sıralı/hepsi eşit
Kabarcık	$\Omega(n^2)$	$O(n^2)$	$\Theta(n^2)$	$O(1)$	en iyi: sıralı/hepsi eşit
Birleşmeli	$\Omega(n \log n)$	$O(n \log n)$	$\Theta(n \log n)$	$O(n)$	alan maliyeti yüksek
Kümeleme	$\Omega(n \log n)$	$O(n \log n)$	$\Theta(n \log n)$	$O(n)$	iki aşama!
Hızlı	$\Omega(n \log n)$	$O(n^2)$	–	$O(\log n)$–$O(n)$	pivot seçimi belirleyici
Radiks	$O(d(n+k))$		$\Theta(d(n+k))$	$O(n+k)$	karşılaştırmasız, kovalar

4 Listeler ve Bağlantılı Listeler

4.1 Veri yapısı ve soyut veri tipi

Bir veri yapısı tanımlarken iki şey belirlenir: (1) **nesneler kümesi** (ne depolanacak: tam sayı, ondalık, karakter, öğrenci kaydı…), (2) **işlemler kümesi** (bu veriler üzerinde hangi işlemler yapılacak). **Soyut veri tipi (SVT) = soyut veri yapısı**: dildeki temel tipleri kullanarak oluşturduğumuz yeni tip (kum + çimento → beton benzetmesi). C++'ta `class` ile tanımlanır.

- **class – struct farkı:** class içinde değişkenler *ve* fonksiyonlar tanımlanabilir (fonksiyonu sadece sınıf içindekiler kullanır); struct'ta fonksiyonlar dışarıda tanımlanır ve başka yerlerden erişilebilir (risk).
- **SVT avantajları:** modülerlik (kütüphane bir kez yazılır), bilgi gizleme, hızlı prototipleme. **Dezavantajı:** verimlilik kaybı (kullanılmayan yetenekler de yüklenir).

4.2 Dizi ile bağlantılı liste karşılaştırması

	Dizi	Bağlantılı liste
Boyut	Sabit, derleme anında verilir	Değişken (bellek elverdiği sürece)
Bellek	Ardışık *blok* hafıza gerekir (6'lık dizi 5'lik boşluklara sığmaz)	Düğümler bellekte dağınık olabilir, bağlantılarla birleşir
Veri tipi	Tek tip, hücrede tek veri (öğrenci için 3 ayrı dizi: no, ad, soyad)	Düğümde birden çok/farklı tipte veri (no, ad, soyad birlikte)
Başa ekleme/silme	Tüm elemanlar kayar: $O(n)$	Sabit: $O(1)$
k. eleman (findKth)	Sabit: $O(1)$	Baştan gezilir: $O(n)$
Bulma, yazdırma	$O(n)$	$O(n)$

4.3 Liste işlemleri (slayt örneği: 34, 12, 52, 16, 12)

- `printList`: listeyi yaz. `makeEmpty`: listeyi boşalt (dönem bitti, yeni ders).
- `find(52)` → 3 (konumu döner; bulunamazsa 0/NULL). Sıralı olmadığı için baştan tek tek bakılır.
- `insert(3, x)` → 34, 12, 52, **x**, 16, 12 (3. veriden sonra ekle; tren vagonu araya eklemek gibi).
- `remove(52)` → 34, 12, 16, 12 (vagonu çıkarıp önceki ve sonrakini bağlamak).
- `findKth(5)` → 12 (5. eleman). *Ders varsayımı: her veriden bir tane vardır.*

4.4 Bağlantılı liste gerçekleştirmesi (C++)

```
class Node {
public:
    double data;      // veri (ogrenci icin no, ad, soyad olabilir)
    Node*  next;      // sonraki dugume isaretci
};
class List {
public:
    List(void) { head = NULL; }             // yapici: bos liste
    ~List(void);                            // yikici
    bool  IsEmpty() { return head == NULL; }
    Node* InsertNode(int index, double x);
    int   FindNode(double x);
    int   DeleteNode(double x);
```

```
    void  DisplayList(void);
private:
    Node* head;       // ilk dugumu gosterir; verisi yok (lokomotif)
};
```

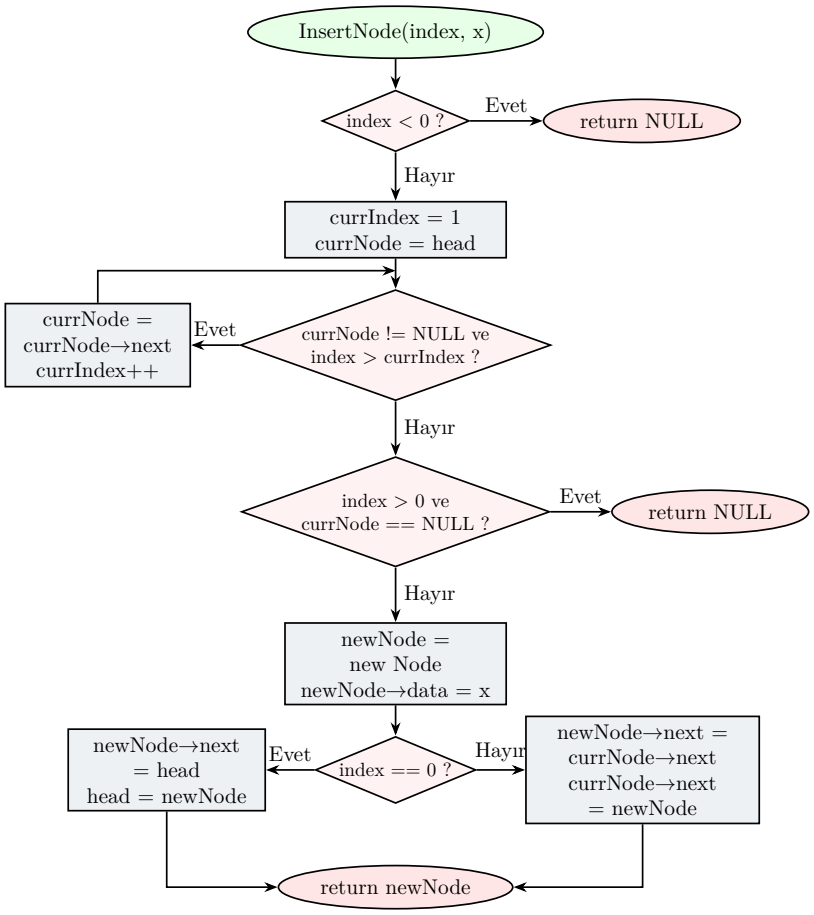
`head` *private*'tır: listenin başlangıcını dışarıdan kimse değiştirememeli. Son düğümün `next`'i NULL'dır.

InsertNode – belirtilen konumdan sonra ekleme

```
Node* List::InsertNode(int index, double x) {
    if (index < 0) return NULL;
    int currIndex = 1;
    Node* currNode = head;
    while (currNode && index > currIndex) {    // index'inci dugumu bul
        currNode = currNode->next;
        currIndex++;
    }
    if (index > 0 && currNode == NULL) return NULL;   // oyle bir dugum yok
    Node* newNode = new Node;
    newNode->data = x;
    if (index == 0) {                          // basa ekle
        newNode->next = head;
        head = newNode;
    } else {                                   // currNode'dan sonra ekle
        newNode->next = currNode->next;
        currNode->next = newNode;
    }
    return newNode;
}
```

Bağlantı sırası

Araya eklemede önce `newNode->next = currNode->next`, *sonra* `currNode->next = newNode` yapılır. Ters sırada yapılırsa listenin geri kalanı kaybolur.



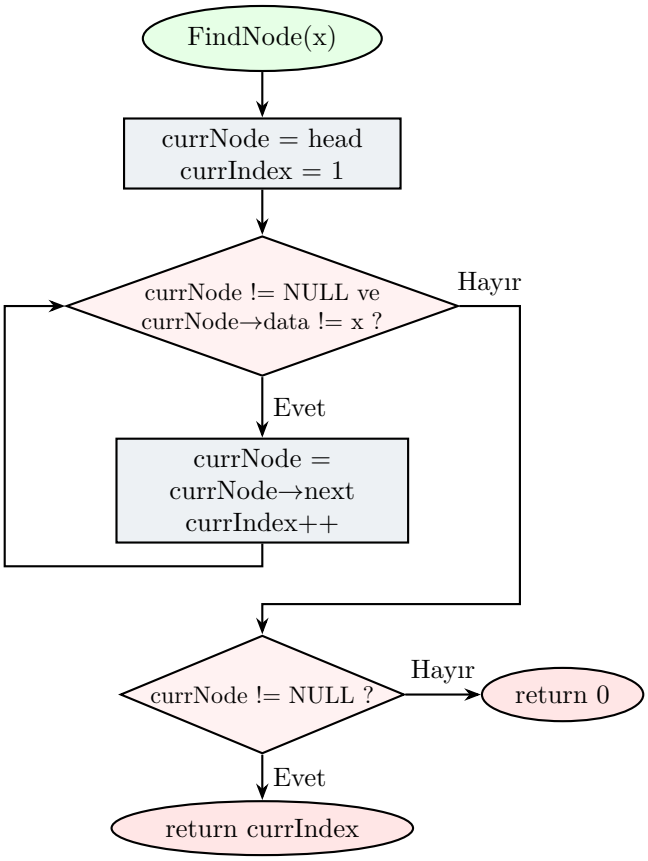
Yürütme zamanı: en kötü (sona ekleme) $T(n) = 3n + 8 \Rightarrow O(n)$; en iyi (başa ekleme) sabit $\Rightarrow \Omega(1)$ (ayrıntı: çözüm dosyası).

FindNode ve DeleteNode

```
int List::FindNode(double x) {
    Node* currNode = head;
    int currIndex = 1;
    while (currNode && currNode->data != x) {
        currNode = currNode->next;
        currIndex++;
    }
    if (currNode) return currIndex;
    return 0;                                   // bulunamadi
}
int List::DeleteNode(double x) {
    Node* prevNode = NULL;
    Node* currNode = head;
    int currIndex = 1;
    while (currNode && currNode->data != x) {
        prevNode = currNode;
        currNode = currNode->next;
        currIndex++;
    }
    if (currNode) {
        if (prevNode) prevNode->next = currNode->next;   // aradan cikar
        else          head = currNode->next;             // bastaki silindi
        delete currNode;
        return currIndex;
```

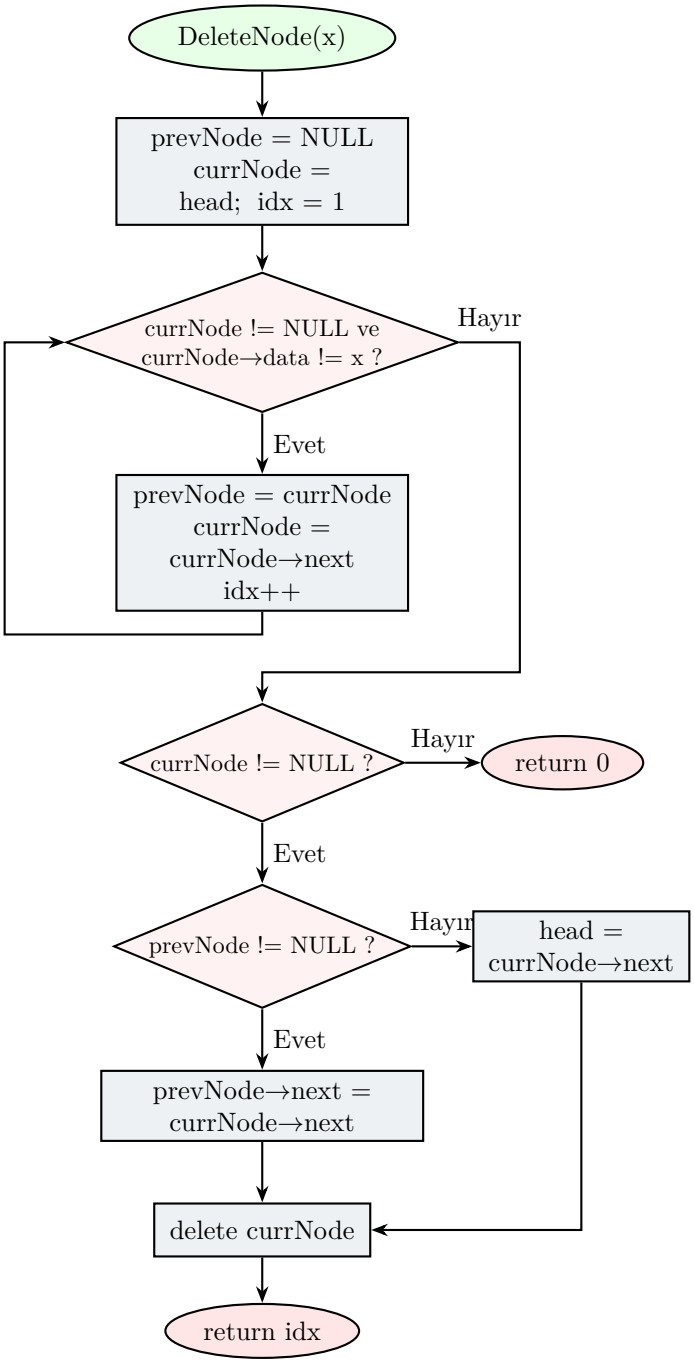
```
    }
    return 0;
}
void List::DisplayList() {
    int num = 0;
    Node* currNode = head;
    while (currNode != NULL) {
        cout << currNode->data << endl;
        currNode = currNode->next;
        num++;
    }
    cout << "Dugum sayisi: " << num << endl;
}
```

Akış diyagramı: FindNode (listede arama)



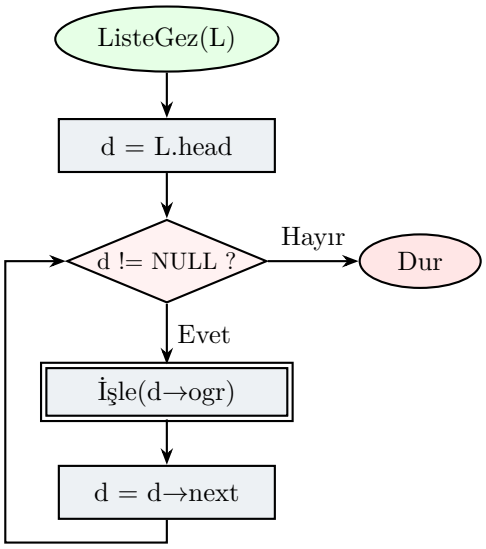
Farz edin ki sorularında listede öğrenci aramanın kalıbıdır.

Akış diyagramı: DeleteNode (listeden silme)



Silinecek düğümün *öncekisi* gerektiği için `prevNode` tutulur; başlaki siliniyorsa `head` güncellenir.

Akış diyagramı: listeyi baştan sona gezme (DisplayList kalıbı)



▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Zaman ve alan karmaşıklığı

İşlem	En iyi	En kötü	Ek alan	Not
`InsertNode` (konum k)	$\Omega(1)$	$O(n)$	$O(1)$	en iyi başa ekleme; en kötü sona ekleme, $T(n) = 3n + 8$
`FindNode`	$\Omega(1)$	$O(n)$	$O(1)$	en iyi aranan ilk düğümde; en kötü sonda veya yok
`DeleteNode`	$\Omega(1)$	$O(n)$	$O(1)$	silinecek düğüm önce aranır; bağlantıyı değiştirmek sabit
`DisplayList`	$\Theta(n)$	$\Theta(n)$	$O(1)$	her düğüm bir kez yazılır
`findKth`(k)	$\Omega(1)$	$O(n)$	$O(1)$	dizide $O(1)$; listede baştan k düğüm gezilir

4.5 Liste çeşitleri

- **Dairesel bağlantılı liste:** son düğüm NULL yerine ilk düğümü gösterir.
- **Çift yönlü bağlantılı liste:** her düğümde `next` ve `prev` vardır; geriye doğru gezilebilir, silmede önceki düğümü aramaya gerek kalmaz.

Sınav bağlantısı

Farz edin ki sorularında liste gezme kalıbı: `d = liste.head; while (d != NULL) { …d = d->next; }`. Hocanın kabul ettiği temel işlemler: `head`, `next`, `FindNode/find`, `InsertNode`, `DeleteNode`. Yılı bilinmeyen bir vizede *"bağlantılı liste ekleme akış diyagramı, karmaşıklığı"* soruldu ve fonksiyonu öğrencinin yazması istendi.

5 Yığın (Stack) ve Kuyruk (Queue)

5.1 Yığın – LIFO (son giren ilk çıkar)

Ekleme ve silme yalnızca **tepeden (top)** yapılır. Temel işlemler: `push(x)` (tepeye ekle), `pop()` (tepedekini çıkar ve döndür), `top()` (tepedekini silmeden gör), `isEmpty()`, `isFull()`.

WhatsApp'ta tespit edilen hata

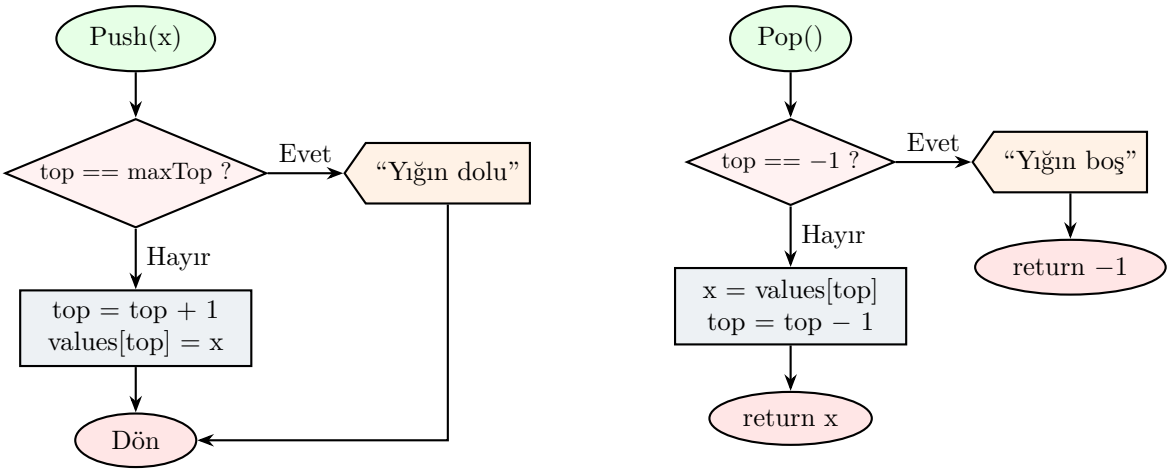
"Drive'daki farz edin ki sorularında stack.push ile if kontrolü yapılmış... Hatalı, veri alırken pop kullanıyoruz." Yığından eleman *almak* için `pop` kullanılır; yığının elemanlarına indisle (`stack[i]`) erişilmez.

Dizi ile gerçekleştirme

```
class Stack {
public:
    Stack(int size = 10) { maxTop = size - 1; values = new double[size]; top = -1; }
    bool IsEmpty() { return top == -1; }        // bos yigin: top = -1
    bool IsFull()  { return top == maxTop; }
    double Push(const double x) {
        if (IsFull()) { cout << "Yigin dolu"; return -1; }
        values[++top] = x;                      // once top artar, sonra yazilir
        return x;
    }
    double Pop() {
        if (IsEmpty()) { cout << "Yigin bos"; return -1; }
        return values[top--];                   // once okunur, sonra top azalir
    }
    double Top() { if (IsEmpty()) return -1; return values[top]; }
private:
    int maxTop;  int top;  double* values;
};
```

Varsayılan boyut (10) kullanıcının boyut vermeyi unutmasına karşı konur. C'de dizinin 0. hücresinde veri olduğu için boş yığın -1 ile gösterilir.

Akış diyagramı: Push ve Pop (dizi ile)



Boş yığında `top = -1`, dolu yığında `top == maxTop`.

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Bağlantılı liste ile gerçekleştirme

`push` = listenin *başına* ekleme (`InsertNode(0,x)`), `pop` = baştaki düğümü silme. Boyut sınırı yoktur. Hoca: *"baştan ekleme silmenin zaman karmaşıklığı sabit, order of C"* [video 10 Liste 2 47:34].

```
class Stack : public List {
public:
    double Top()  { if (head == NULL) return -1; return head->data; }
    void   Push(const double x) { InsertNode(0, x); }
    double Pop() {
        if (head == NULL) { cout << "Yigin bos"; return -1; }
        double val = head->data;
        DeleteNode(val);                        // bastaki dugum silinir
        return val;
    }
};
```

Karmaşıklık: push, pop, top: $O(1)$.

5.2 Postfix (sonek) ifadenin yığınla hesaplanması

Soldan sağa okunur: **sayı** gelince `push`; **işlem** gelince iki kez `pop` – *ilk çıkan sağ işlenen*, ikinci çıkan sol işlenen – sonuç `push`. Sonda yığında kalan değer sonuçtur.

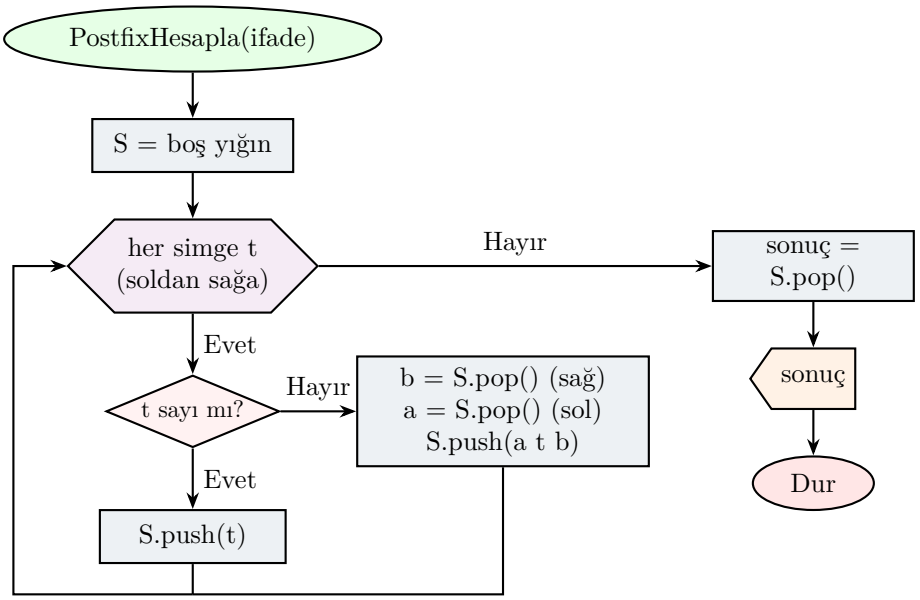
Okunan	İşlem	Yığın (alt→tepe)
5, 6, 8	push	5, 6, 8
*	$6 \times 8 = 48$	5, 48
+	$5 + 48 = 53$	53
7, 6, 2	push	53, 7, 6, 2
/	$6/2 = 3$	53, 7, 3
–	$7 - 3 = 4$	53, 4
3	push	53, 4, 3
*	$4 \times 3 = 12$	53, 12
–	$53 - 12 = 41$	**41**

(Hocanın örnek sınav sorusu: `5 6 8 * + 7 6 2 / - 3 * -` = 41.) Bölme ve çıkarmada işlenen sırası önemlidir: *"7/5 mi yapacaksın yoksa 5/7 mi?"*

```
PostfixHesapla(ifade)
  S = bos yigin
  for (her simge t, soldan saga)
     if (t sayi) S.push(t)
     else
        b = S.pop()     // sag
        a = S.pop()     // sol
        S.push(a t b)
  Yaz(S.pop())
```

2021 vizesi ve örnek sınav sorularında soruldu. Çıkarma ve bölmede ilk çıkan *sağ* işlenendir. Her simge bir kez işlenir: $\Theta(n)$.

Akış diyagramı: sonek (postfix) ifadenin yığınla hesaplanması



▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

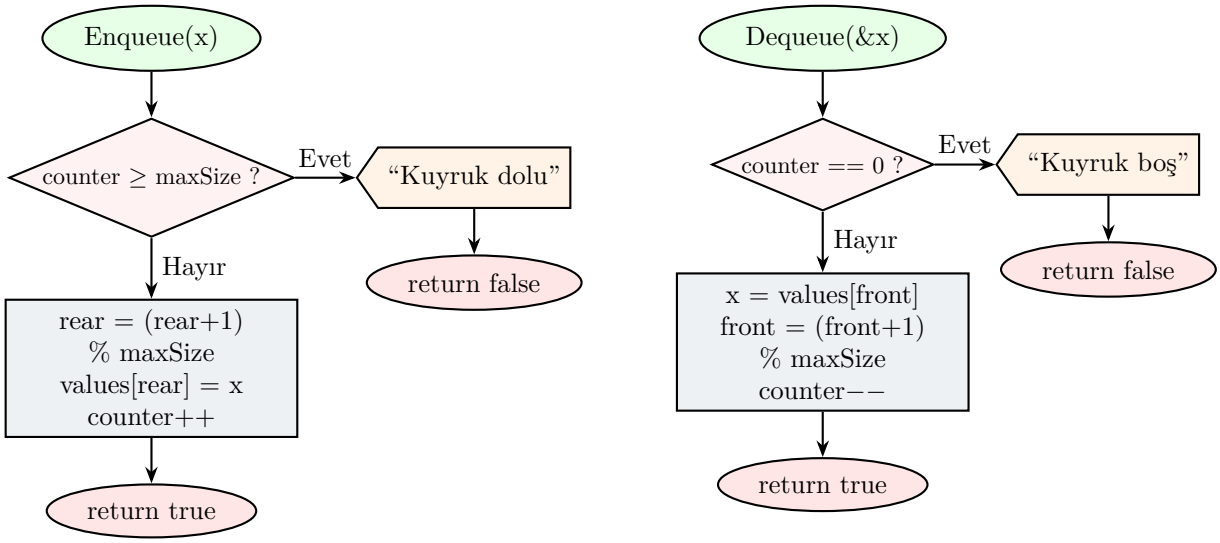
5.3 Kuyruk – FIFO (ilk giren ilk çıkar)

Ekleme **arkadan (rear/back)**, silme **önden (front)**. Temel işlemler: `enqueue(x)`, `dequeue()`, `isEmpty()`, `isFull()`, (`size()`).

Dizi ile dairesel kuyruk

```
class Queue {
public:
    Queue(int size = 10) { values = new double[size]; maxSize = size;
                           front = 0; rear = -1; counter = 0; }
    bool IsEmpty() { return counter == 0; }
    bool IsFull()  { return counter >= maxSize; }
    bool Enqueue(double x) {
        if (IsFull()) { cout << "Kuyruk dolu"; return false; }
        rear = (rear + 1) % maxSize;            // dairesel ilerleme
        values[rear] = x;  counter++;
        return true;
    }
    bool Dequeue(double& x) {
        if (IsEmpty()) { cout << "Kuyruk bos"; return false; }
        x = values[front];
        front = (front + 1) % maxSize;
        counter--;
        return true;
    }
private:
    int front, rear, counter, maxSize;  double* values;
};
```

Akış diyagramı: Enqueue ve Dequeue (dizi ile dairesel)



`counter` eleman sayısını tutar; `% maxSize` ile indis başa döner.

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Dairesel kuyrukta enqueue ve dequeue $O(1)$. Dairesel olmayan, silmede tüm elemanları öne kaydıran dizi kuyruğunda silme $n-1$ kaydırma $\Rightarrow O(n)$ [video 11 36:04]. Bağlantılı listeyle: enqueue sona (son düğüm işaretçisi tutularak), dequeue baştan, ikisi de $O(1)$.

5.4 Farz edin ki sorularında yığın ve kuyruğu bozmadan gezmek

İki temel kalıp – Not2 (“sınıf listeleri değişmeyecek”) için şart

Yığın: geçici yığına aktar, işini yap, geri aktar (iki kez ters çevrildiği için sıra korunur):

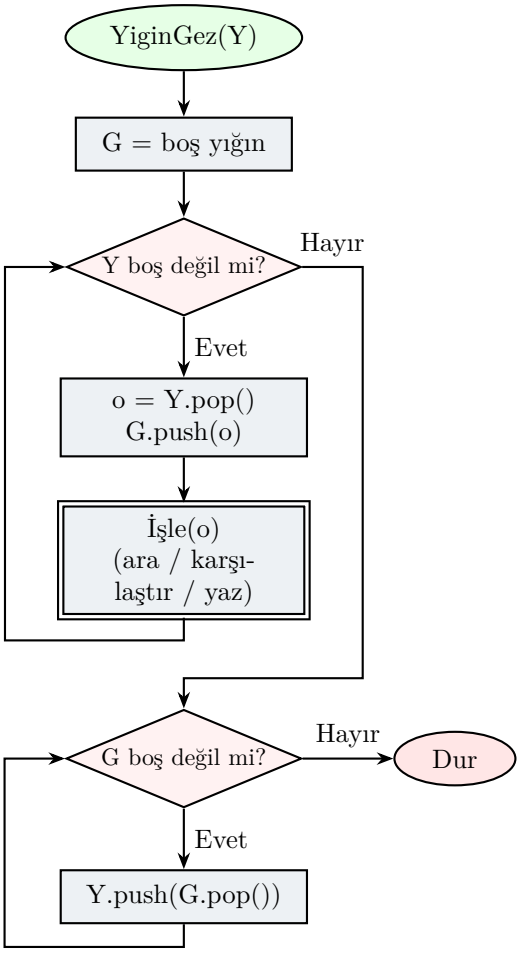
```
Stack gec
while (!Y.isEmpty())  { o = Y.pop();  gec.push(o);  ...islem... }
while (!gec.isEmpty())  Y.push(gec.pop())
```

Kuyruk: boyutu kadar döndür (çıkar–işle–arkaya geri koy); tur bitince aynı sıra:

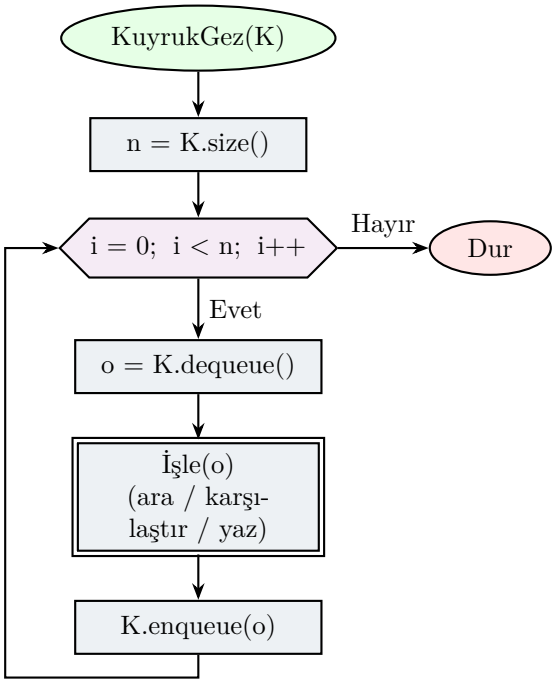
```
n = K.size()
for (i = 0; i < n; i++)  { o = K.dequeue();  ...islem...;  K.enqueue(o); }
```

Hoca derste ayrıca `yigin.clone()`, `kuyruk.clone()` ile kopya üzerinde çalıştı. Kopya fonksiyonu kullanacaksanız ne yaptığını açıklayın; geçici yapı ile geri yükleme her durumda kabul edilir.

Akış diyagramı: yığını bozmadan gezme



Akış diyagramı: kuyruğu döndürerek gezme



5.5 Zaman ve alan karmaşıklığı

İşlem	En iyi	En kötü	Ek alan	Not
`push` / `pop` / `top`	$\Omega(1)$	$O(1)$	$O(1)$	dizi ve bağlantılı liste gerçekleştirmesinde de sabit
`enqueue` / `dequeue` (dairesel)	$\Omega(1)$	$O(1)$	$O(1)$	`% maxSize` ile indis başa döner
`dequeue` (kaydırmalı dizi)	$\Omega(1)$	$O(n)$	$O(1)$	silmede $n-1$ eleman öne kayar
Postfix hesaplama	$\Theta(n)$	$\Theta(n)$	$O(n)$	her simge bir kez; yığında en fazla n sayı
Bozmadan gezme (yığın/kuyruk)	$\Theta(n)$	$\Theta(n)$	$O(n)$ / $O(1)$	yığında geçici yığın $O(n)$; kuyrukta döndürme ek alan istemez

6 İkili Ağaçlar ve İkili Arama Ağaçları

6.1 Neden ağaç?

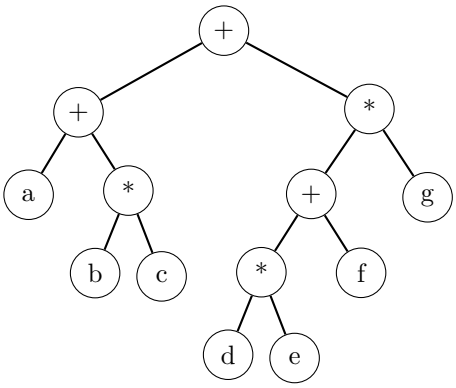
Bağlantılı listede arama, ekleme, silme doğrusaldır ($O(n)$). Hedef: bu işlemleri $O(\log n)$'e indiren bir yapı. Ağaç düğümlerden oluşur: **kök** (ebeveyni yok), **çocuk/ebeveyn**, **yaprak** (çocuğu yok), **kardeş** (aynı ebeveyn), **yol uzunluğu** (kenar sayısı), **derinlik** (kökten düğüme yol), **yükseklik** (düğümden en uzak yaprağa yol; yapraklar 0, ağacın yüksekliği = kökün yüksekliği). **İkili ağaç:** her düğümün en fazla iki çocuğu vardır.

6.2 İfade ağaçları

Yapraklar sayı/değişken, iç düğümler işlemdir. *"Burada önemli husus: sayılar kesinlikle yaprakta olacak"* [video 13 00:03]. Toplama/çarpmada sol–sağ yer değiştirebilir ama *çıkarma ve bölmede sıra önemlidir.*

- **Sonek ifadesinden ağaç:** soldan sağa okunur; işlenenler beklenir, işlem gelince son iki alt ağaç onun sol ve sağ çocuğu olur (yığın mantığı). Örnek `a b c * + d e * f + g * +`: `bc*`→`a+(b*c)`→`de*`→`(d*e)+f`→`((d*e)+f)*g`→ kök `+`.
- **Eşitlikten ağaç:** parantez ve işlem önceliğine dikkat edilerek en içten dışa doğru alt ağaçlar kurulur.

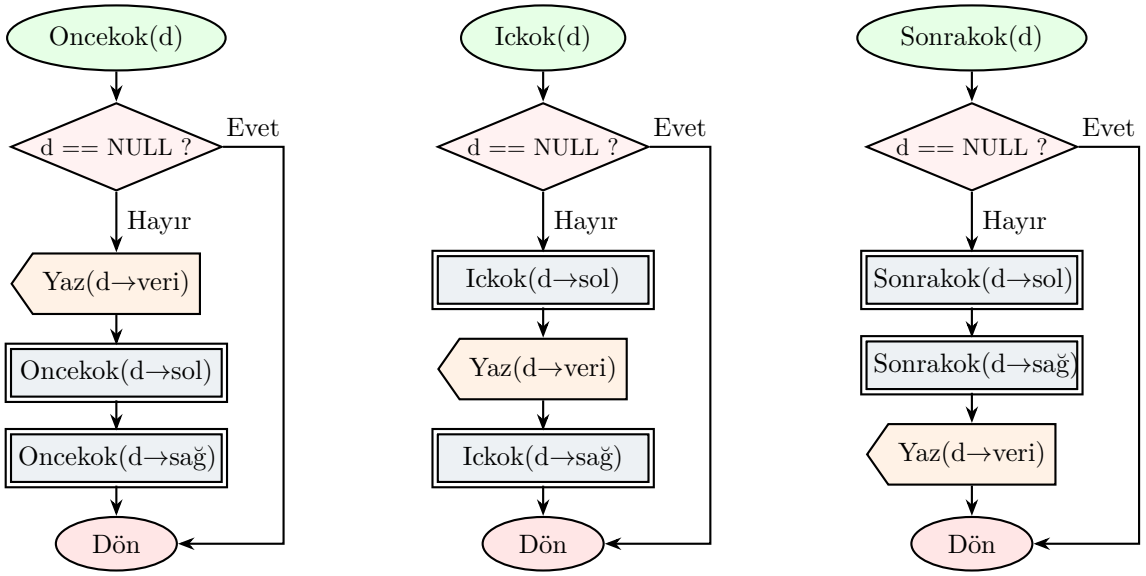
6.3 Ağaç gezme (traversal)



Gezme	Sıra	Sonuç
Öncekök (preorder)	kök, sol, sağ	`+ + a * b c * + * d e f g` (önek)
İçkök (inorder)	sol, kök, sağ	`a + b * c + d * e + f * g` (içek)
Sonrakök (postorder)	sol, sağ, kök	`a b c * + d e * f + g * +` (sonek)

```
Oncekok(d):  if d == NULL return;  Yaz(d);  Oncekok(d->sol);  Oncekok(d->sag)
Ickok(d):    if d == NULL return;  Ickok(d->sol);  Yaz(d);  Ickok(d->sag)
Sonrakok(d): if d == NULL return;  Sonrakok(d->sol);  Sonrakok(d->sag);  Yaz(d)
```

Akış diyagramı: öncekök, içkök, sonrakök



Üç diyagram aynıdır; sadece "Yaz" kutusunun yeri değişir: başta (kök–sol–sağ), ortada (sol–kök–sağ), sonda (sol–sağ–kök).

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

"Sol sağ kök. Solun sağa her zaman önceliği var" [video 12 20:19]. Her gezme her düğümü bir kez ziyaret eder: $\Theta(n)$. 2025 vizesinde sorulan "sol taraf → sağ taraf → düğüm" gezmesi sonrakök'tür.

6.4 İkili arama ağacı (İAA)

Kural: her X düğümü için sol alt ağaçtaki tüm anahtarlar X'ten küçük, sağ alt ağaçtakiler büyüktür – *"bu ikili ağacın her aşamasında, her kademede geçerli olacak"*. Harflerde alfabetik önde olan soldadır. Eşit değer için bir kural belirlenip (ör. sağa) ona sadık kalınır.

- **Arama:** kökten başla; aranan küçükse sola, büyükse sağa; NULL'a gelinirse yok. Dengeli ağaçta $O(\log n)$, ip gibi dizilmiş ağaçta $O(n)$.
- **En küçük / en büyük:** sürekli en sola / en sağa gidilir (*"NULL'ı gördüğüm yerdeki bir önceki eleman en küçük"*).
- **İçkök gezme sıralı listeyi verir** – ayrıca sıralama algoritması gerekmez.
- **Ekleme:** arama gibi inilir, NULL bulunan yere yaprak olarak eklenir. Sıralı gelen veri (1,2,3,…) ağacı sağa doğru "ip gibi" uzatır.
- **Silme (hocanın kuralları):**
  1. Yaprak ⇒ doğrudan silinir.
  2. Tek çocuklu ⇒ çocuğu, silinen düğümün ebeveynine (dedeye) bağlanır.
  3. İki çocuklu ⇒ **sağ alt ağacın en küçüğü (minimumu) silinen düğümün yerine geçer** [video 12 43:31]; sonra o eleman eski yerinden (1. veya 2. kurala göre) silinir.

! Hoca derste vurguladı WhatsApp aktarımı

"AVL'de kök sildikten sonra yeni kökü nasıl seçiyorduk? – Sağ alt ağacın minimumu." "İkisi de mantıken doğru ama biz sağın en küçüğünü alarak işleyeceğiz dedi hoca." "Listeyi ikili arama

ağacına yerleştirirken... yine de tüm adımları tek tek gösterin dedi."

Örnek: KAR, MOR, BAL, ŞEN, ÜZÜM, TEK, CAN, ÖZ ekle; kökü sil; DAL ekle

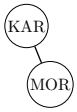
▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Ekleme adımları (küçükler sola, büyükler sağa; her kademede kontrol):

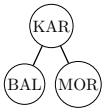
KAR eklendi



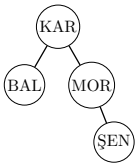
MOR eklendi



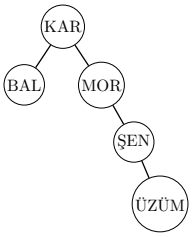
BAL eklendi



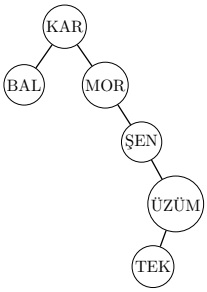
ŞEN eklendi



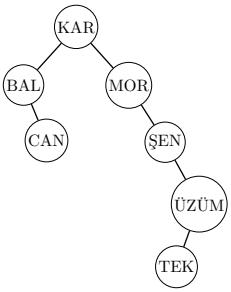
ÜZÜM eklendi



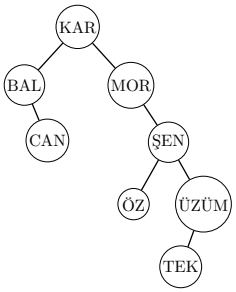
TEK eklendi



CAN eklendi

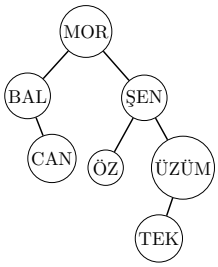


ÖZ eklendi

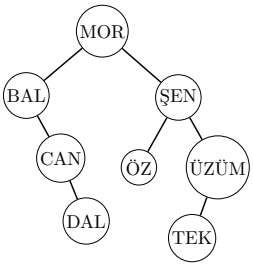


1. kök silme (**KAR**)

KAR iki çocuklu: sağ alt ağacın en küçüğü MOR yerine geçer.



DAL eklendi



6.5 İAA kodları

```
Node* Ara(Node* d, x)
   if (d == NULL || d->veri == x) return d
   if (x < d->veri) return Ara(d->sol, x)
   return Ara(d->sag, x)

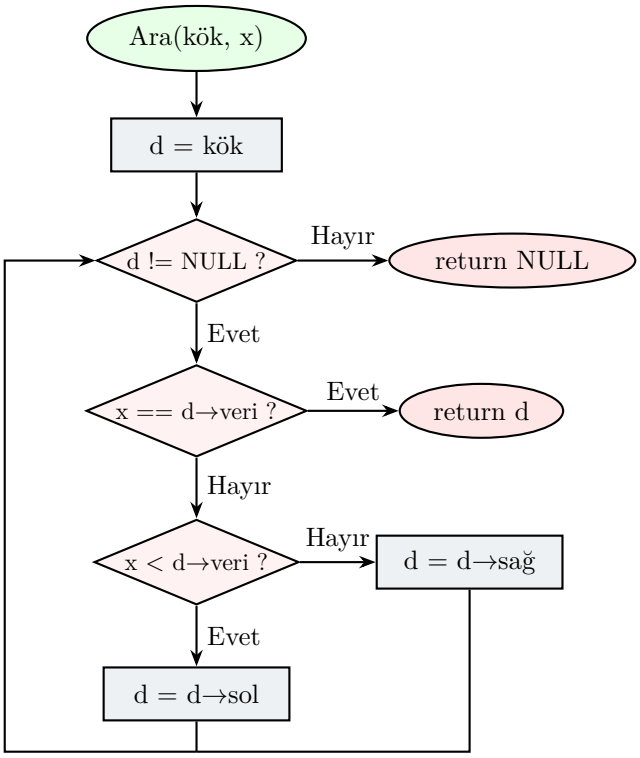
Node* Ekle(Node* d, x)
   if (d == NULL) return YeniDugum(x)
   if (x < d->veri)  d->sol = Ekle(d->sol, x)
```

```
    else              d->sag = Ekle(d->sag, x)
    return d

Node* EnKucuk(Node* d)   while (d->sol != NULL) d = d->sol;   return d
```

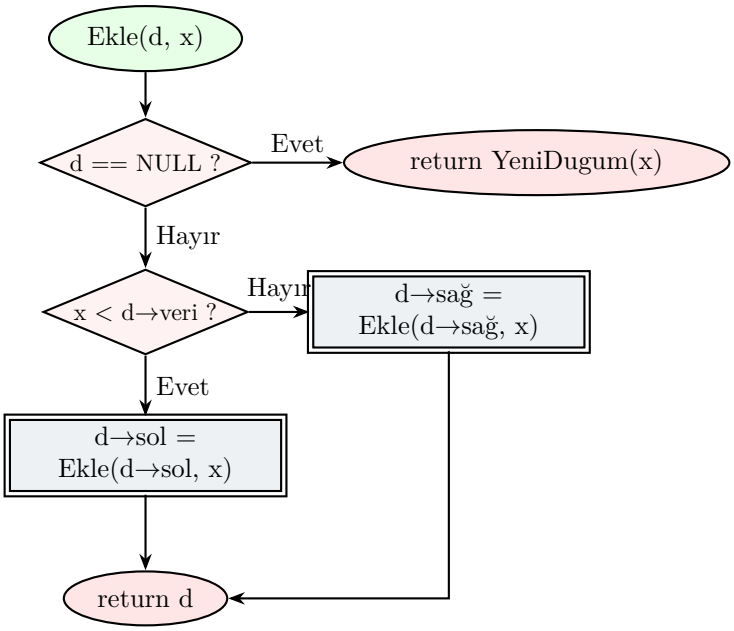
Akış diyagramı: ikili arama ağacında arama (hocanın ödevi)

Hocanın derste verdiği soru: *"ikili ağaçta verilen bir sayıyı arayan fonksiyonun akış diyagramı"*.



Akış diyagramı: ikili arama ağacına ekleme

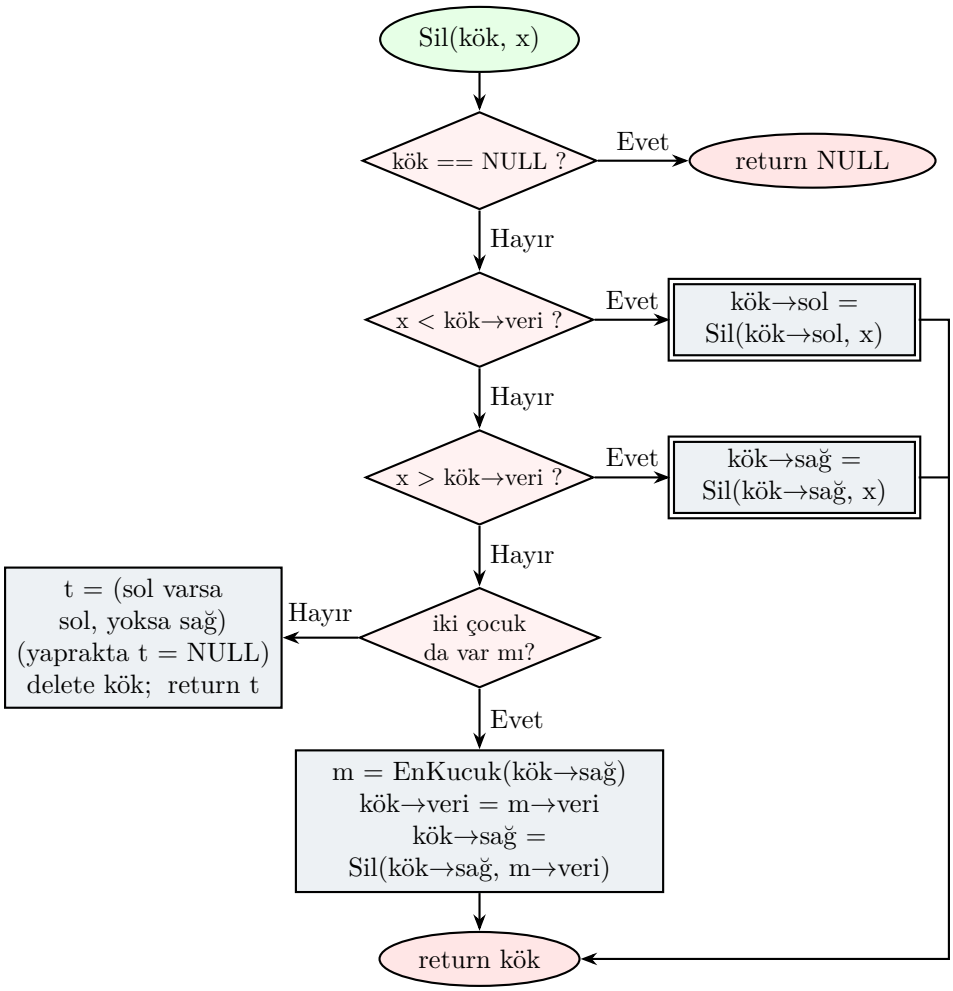
Eşit değerler için seçilen kural (burada: sağa) sonuna kadar uygulanır.



Akış diyagramı: ikili arama ağacından silme (2025–26 vize sorusu)

```
Sil(kok, x)
  if (kok == NULL) return NULL
  if (x < kok->veri)
     kok->sol = Sil(kok->sol, x)
  else if (x > kok->veri)
     kok->sag = Sil(kok->sag, x)
  else if (iki cocugu var)
     m = EnKucuk(kok->sag)
     kok->veri = m->veri
     kok->sag = Sil(kok->sag, m->veri)
  else            // yaprak / tek cocuk
     t = (sol varsa sol, yoksa sag)
     delete kok;  return t
  return kok
```

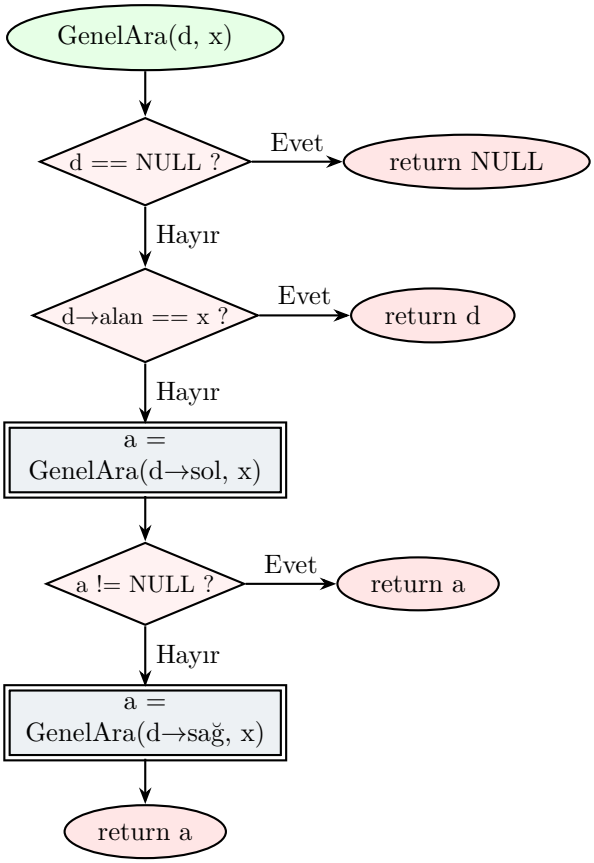
Hocanın kuralı: iki çocukluda *sağ alt ağacın en küçüğü* yerine geçer. Satır satır yürütme zamanı çözümler dosyasındadır (2025–26 vize).



Genel (arama olmayan) ikili ağaçta arama – hocanın tahtada yazdığı

Ağaç aranan alana göre indekslenmemişse (ör. AVL numaraya göre, biz ada göre arıyoruz) hem sol hem sağ alt ağaca bakılır: `if (d==NULL) return NULL; if (d->ad==x) return d; a = Ara(d->sol,x); if (a!=NULL) return a; return Ara(d->sag,x);` – $O(n)$.

Akış diyagramı: indekslenmemiş alana göre ağaçta arama



6.6 Zaman ve alan karmaşıklığı

h: ağacın yüksekliği. Dengeli ağaçta $h \approx \log_2 n$, ip gibi dizilmiş (sıralı veriyle oluşmuş) ağaçta $h = n - 1$.

İşlem	En iyi	En kötü	Ek alan	Not
Arama	$\Omega(1)$	$O(n)$	$O(1)$ / $O(h)$	en iyi kökte; $O(h)$, dengeli ise $O(\log n)$; özyinelemeli sürüm h kadar çağrı
Ekleme	$\Omega(1)$	$O(n)$	$O(h)$	NULL bulunana kadar inilir: $O(h)$
Silme	$\Omega(1)$	$O(n)$	$O(h)$	arama + (iki çocukluda) sağ alt ağacın en küçüğünü bulma: $O(h)$
En küçük / en büyük	$\Omega(1)$	$O(n)$	$O(1)$	sürekli sola / sağa: $O(h)$
Gezmeler (öncekök, içkök, sonrakök)	$\Theta(n)$	$\Theta(n)$	$O(h)$	her düğüm bir kez ziyaret edilir
İndekslenmemiş alanda arama	$\Omega(1)$	$O(n)$	$O(h)$	sol/sağ seçimi yapılamaz, iki alt ağaca da bakılır

7 AVL Ağaçları

7.1 Neden AVL?

İkili arama ağacına veriler sıralı gelirse ağaç bağlantılı listeye döner, yükseklik n olur ve arama/ekleme/silme $O(n)$'e çıkar. AVL ağacı bir **ikili arama ağacıdır** (İAA'nın bütün kuralları – ekleme, silme, gezme – aynen geçerli) ve buna bir **denge kuralı** ekler; amaç yüksekliği $O(\log n)$'de tutarak işlemleri $O(\log n)$ garanti etmektir (*"terazinin iki kefesi gibi"*).

7.2 Denge kuralı

- Bir düğümün yüksekliği = çocuklarının en büyük yüksekliği + 1 (boş ağaç -1, yaprak 0).
- **Her düğümde sol ve sağ alt ağaçların yükseklik farkı en fazla 1'dir.**

! Hoca derste vurguladı [video 13 AVL 1 45:18]

"Denge ne zaman bozulur? Aradaki fark 2 olduğu zaman bozulur. Aradaki fark asla 2'den fazla olmaz. Eğer 3, 4, 5 oluyorsa o ağaç tamamen yanlış demektir... 2'yi görünce düzeltmezsem o ağaç yanlış olur. ***İstersek geri kalan işlemleri doğru yapalım fark etmez. O ağaç yanlış, puan alamaz.****"*

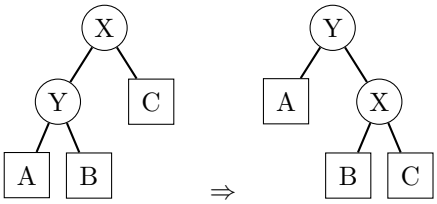
Kontrol *eklenen (veya silinen) düğümden köke doğru*, aşağıdan yukarı yapılır; dengesizliğin ilk görüldüğü (en alttaki) düğüm **problemli düğüm**dür. *"Problem olan düğümün tespiti önemli."*

7.3 Dört problem ve çözümleri

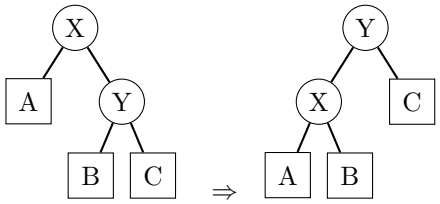
Problemli düğüm X; problemin türü X'ten sonraki *ilk iki dala* bakılarak belirlenir (öğrenci notu: *"3. dallanmaya bakmıyoruz, sorun olan kökten sonraki dallar önemli"*).

Problem	Tanım	Çözüm
Sol–Sol	X'in solunun *sol* tarafı uzun	X'ten **sağa tek dönüş**
Sağ–Sağ	X'in sağının *sağ* tarafı uzun	X'ten **sola tek dönüş**
Sol–Sağ	X'in solunun *sağ* tarafı uzun	çift dönüş: önce X'in solundan **sola**, sonra X'ten **sağa**
Sağ–Sol	X'in sağının *sol* tarafı uzun	çift dönüş: önce X'in sağından **sağa**, sonra X'ten **sola**

Sol–Sol: X'ten sağa tek dönüş

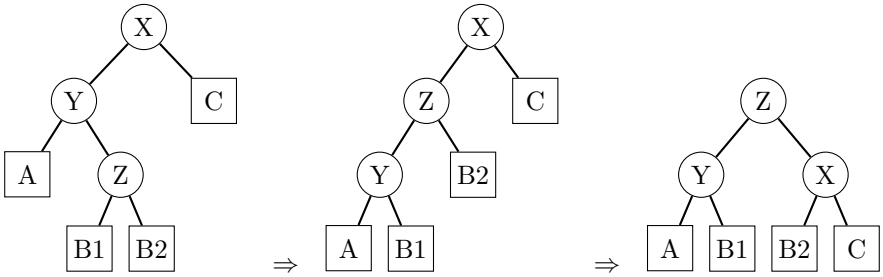


Sağ–Sağ: X'ten sola tek dönüş



B alt ağacı *"Y'den büyük, X'ten küçük olan bölgeye"* taşınır (sol–solda X'in soluna, sağ–sağda X'in sağına).

Sol–Sağ: önce Y'den sola, sonra X'ten sağa (Z yukarı çıkar)



Sağ–Sol bunun ayna görüntüsüdür. Kısa yol: çift dönüşte *ortadaki değer (Z)* köke çıkar, küçük olan sola, büyük olan sağa geçer; Z'nin alt ağaçları (B1, B2) aralıklarına göre dağıtılır.

7.4 Ekleme

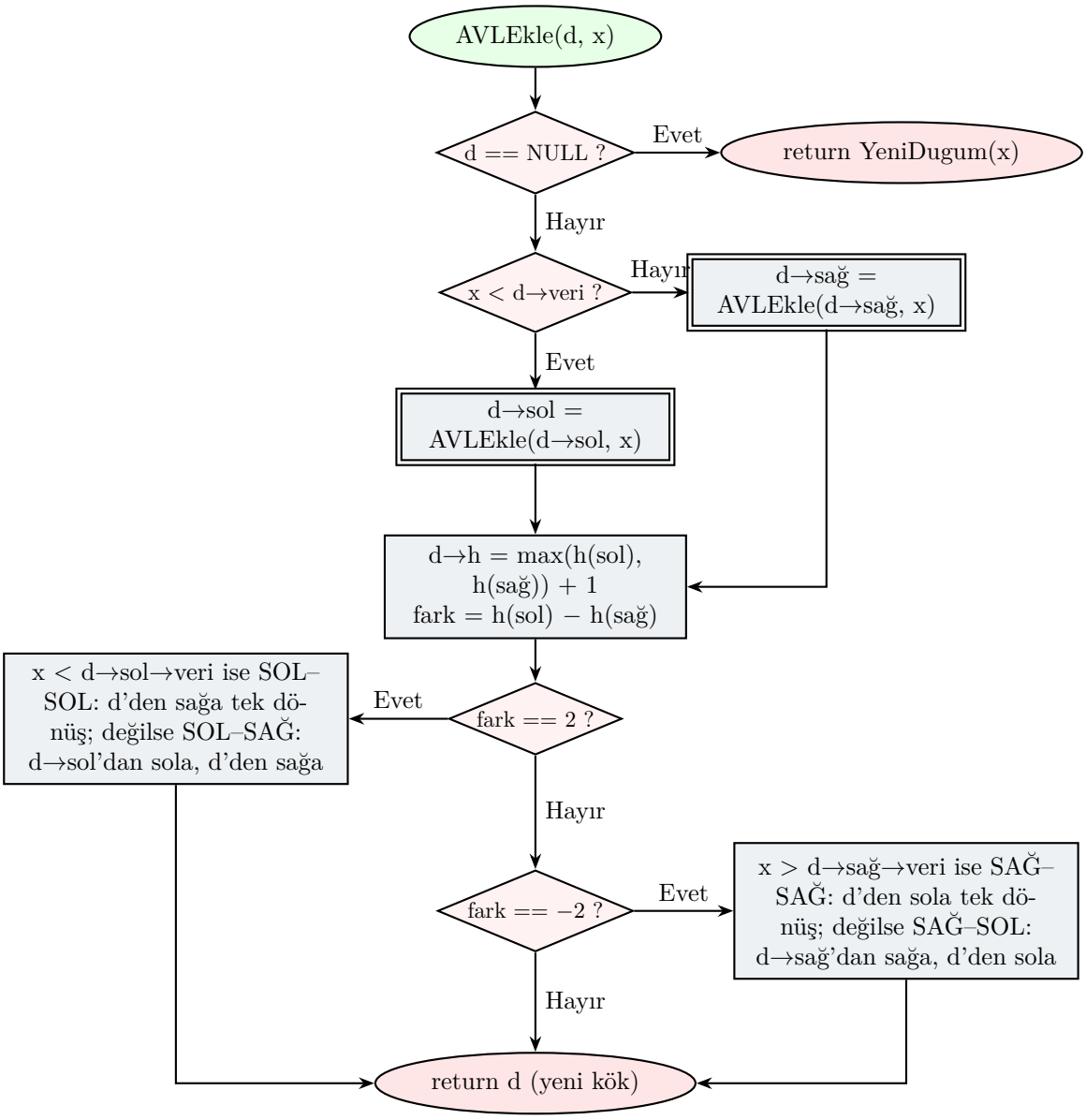
İAA'daki gibi eklenir; eklenen yaprakdan köke doğru her düğümde denge kontrol edilir; bozulma varsa ilgili dönüş yapılır. *"Her eklemede AVL ağacı özelliğini kontrol ediyoruz. Eğer denge bozulduysa dengeyi düzeltmeden bir sonraki adıma geçmiyoruz"* [video 14 25:06]. Eklemede tek bir dönüş (tek veya çift) ağacı dengeler.

Akış diyagramı: AVL ağacına ekleme (denge kontrolü ile)

```
AVLEkle(d, x)
  if (d == NULL) return YeniDugum(x)
  if (x < d->veri)
     d->sol = AVLEkle(d->sol, x)
  else
     d->sag = AVLEkle(d->sag, x)
  d->h = max(h(sol), h(sag)) + 1
  fark = h(sol) - h(sag)
  if (fark == 2)
     if (x < d->sol->veri)        // SOL-SOL
        d = SagaDondur(d)
     else                         // SOL-SAG
        d->sol = SolaDondur(d->sol)
        d = SagaDondur(d)
  else if (fark == -2)
     if (x > d->sag->veri)        // SAG-SAG
        d = SolaDondur(d)
     else                         // SAG-SOL
        d->sag = SagaDondur(d->sag)
        d = SolaDondur(d)
  return d

SagaDondur(x)                     // tek donus
  y = x->sol;  x->sol = y->sag
  y->sag = x;  yukseklikleri guncelle
  return y
```

Özyinelemeden dönerken her düğümde denge kontrolü yapılır (eklenen düğümden köke).



Derste çözülen örnek: 3, 2, 1, 4, 5, 6, 7, 16, 15, 14 ekle; kökü sil

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Ekleme adımları (her eklemeden sonra eklenen düğümden köke doğru denge kontrolü):

3 eklendi
denge var



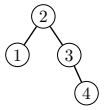
2 eklendi
denge var



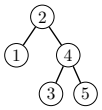
1 eklendi
3 düğümünde denge bozuldu: SOL-SOL problemi ⇒ 3'den sağa tek dönüş.



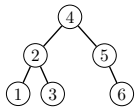
4 eklendi
denge var



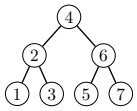
5 eklendi
3 düğümünde denge bozuldu: SAĞ-SAĞ problemi ⇒ 3'den sola tek dönüş.



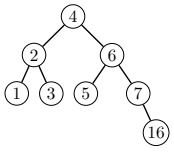
6 eklendi
2 düğümünde denge bozuldu: SAĞ-SAĞ problemi ⇒ 2'den sola tek dönüş.



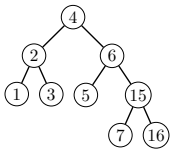
7 eklendi
5 düğümünde denge bozuldu: SAĞ-SAĞ problemi ⇒ 5'den sola tek dönüş.



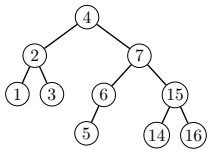
16 eklendi
denge var



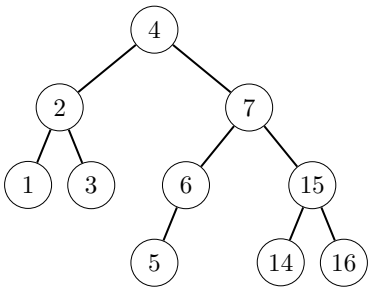
15 eklendi
7 düğümünde denge bozuldu: SAĞ-SOL problemi ⇒ çift dönüş: önce 16'den sağa, sonra 7'den sola.



14 eklendi
6 düğümünde denge bozuldu: SAĞ-SOL problemi ⇒ çift dönüş: önce 15'den sağa, sonra 6'den sola.

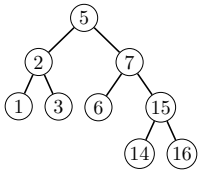


Ağacın son hali (kontrol: içkök gezme sıralı verir: 1, 2, 3, 4, 5, 6, 7, 14, 15, 16):



Silme adımları (iki çocuklu düğüm silinirken *sağ alt ağacın en küçüğü* yerine geçer; ardından denge kontrolü):

1. kök silme: **4**
4 iki çocuklu: sağ alt ağacın en küçüğü 5 köke/yerine geçer. Ardından 5 sağ alt ağaçtaki eski yerinden silinir: 5 yaprak, doğrudan silinir. Denge bozulmadı.



İçkök (inorder: sol–kök–sağ) gezme (son ağaç üzerinde): **1, 2, 3, 5, 6, 7, 14, 15, 16**

7.5 Silme

1. İAA silme kuralları: yaprak → sil; tek çocuk → dedeye bağla; iki çocuk → **sağ alt ağacın en küçüğü** yerine geçer.
2. Silinen yerden köke doğru denge kontrolü; aynı dört problem ve aynı dönüşler. Silmede birden fazla seviyede dönüş gerekebilir.
3. **Hocanın özel kuralı:** problemli düğümün altında hem sol–sol hem sol–sağ (veya hem sağ–sağ hem sağ–sol) *eşit uzunluktaysa "çift dönüşlü olanı tercih ediyoruz"* [video 14 38:38]. (Klasik kaynaklarda bu durumda tek dönüş yapılır; çift dönüş sonucu da dengeli çıkıyorsa hocanın tercihini uygulayın, çıkmıyorsa tek dönüş yapın ve bunu not edin.)

Sınavda sık yapılan hatalar

- "İki kez kök silme"de ikinci silme *yeni oluşan ağacın kökü* üzerinden yapılır. 2025 vizesinde: *"bazılarında denge bozuldu bazılarında bozulmadı"*, *"en sağda XOR vardı, sola döndürmen lazımdı"* – silmeden sonra sağ alt ağacın derinlerinde de kontrol edin.
- Kelimeleri sayıya çevirmek yasak; *"harfleri öğrenip çözmek daha kolay"*.
- Gezme (öncekök/sonrakök) sorusunda hangi ağaç üzerinde istendiğine dikkat edin ("a

şıkkında oluşturduğunuz" = silmeden önceki ağaç).

- Sonuçta içkök gezme sıralı listeyi vermelidir – kendi kontrolünüz için kullanın.

Zaman kazanma

"Bütün ağacı çizmek çok vakit alırsa... değişmeyecek alt ağaca 1 numara deyip aşağıda da 1 numara diye gösterebilirsiniz" [video 14 01:08]; ama *"en sonda son halini görmeyi beklerim"*. Alt ağacın yüksekliğini yanına not edin. AVL sonuçlarınızı `cs.usfca.edu/~galles/visualization/AVLtree.html` ile doğrulayabilirsiniz (öğrenci önerisi; site silmede solun en büyüğünü kullanabilir, dikkat).

Karmaşıklık: yükseklik $\approx 1.44 \log_2 n$; arama, ekleme, silme $O(\log n)$; dönüş $O(1)$.

Zaman ve alan karmaşıklığı

İşlem	En iyi	En kötü	Ek alan	Not
Arama	$\Omega(1)$	$O(\log n)$	$O(1)$	yükseklik her zaman $O(\log n)$; İAA'nın $O(n)$ en kötüsü oluşmaz
Ekleme	$\Omega(\log n)$	$O(\log n)$	$O(\log n)$	yaprağa kadar inilir, köke kadar denge kontrolü; en fazla bir (tek/çift) dönüş
Silme	$\Omega(\log n)$	$O(\log n)$	$O(\log n)$	birden fazla seviyede dönüş gerekebilir; her dönüş $O(1)$
Tek / çift dönüş	$\Theta(1)$	$\Theta(1)$	$O(1)$	sabit sayıda işaretçi değişir
Gezmeler	$\Theta(n)$	$\Theta(n)$	$O(\log n)$	her düğüm bir kez

8 “Farz Edin Ki” Soruları – Çözüm Rehberi

Her sınavda (vize, final, büt) bir soru, **25 puan**. Birkaç ders, farklı veri yapılarında tutulur; belirli bir öğrenci kümesini listeleyen fonksiyon istenir: a) kaba kod, b) mantık, c) akış diyagramı, d) yürütme zamanı ve zaman karmaşıklığı.

Ölçülen şey

Öğrenci yorumu (doğru tespit): *“Bu soruda asıl ölçülen şey istenen veri yapılarının doğru işlemlerinin bilinip bilinmediği. Örneğin stackte push pop kullanmayı bilip bilmediği. Temel işlemleri kullanarak çok basit bir algoritma izlenmesi gerekiyor. Fakat bunu doğru şekilde yapmak ve açıklamak.”*

8.1 Temel işlemler tablosu

Yapı	Kullanılacak işlemler	Bir elemanı bulmak
Bağlantılı liste	`d = L.head; while(d != NULL){…; d = d->next;}`, `FindNode(x)`, `InsertNode`, `DeleteNode`	$O(n)$ (sırasız)
Yığın	`push`, `pop`, `top`, `isEmpty` (+ geçici yığın)	$O(n)$
Kuyruk	`enqueue`, `dequeue`, `isEmpty`, `size` (+ döndürme)	$O(n)$
İAA / AVL	`Ara(anahtar)` / `find`, `Ekle`, `Sil`, içkök gezme	indeks alanında $O(\log n)$ (AVL), İAA en kötü $O(n)$; başka alanda $O(n)$
B+	`search(anahtar)`, `insert`, `delete`, yaprakları soldan sağa gezme	indeks alanında $O(\log n)$

İndeksleme alanına dikkat: AVL “numaraya göre” indeksliyse numarayla aramak $\log n$’dir; adla aramak gerekirse tüm ağaç gezilir ($O(n)$).

8.2 Küme işlemleri

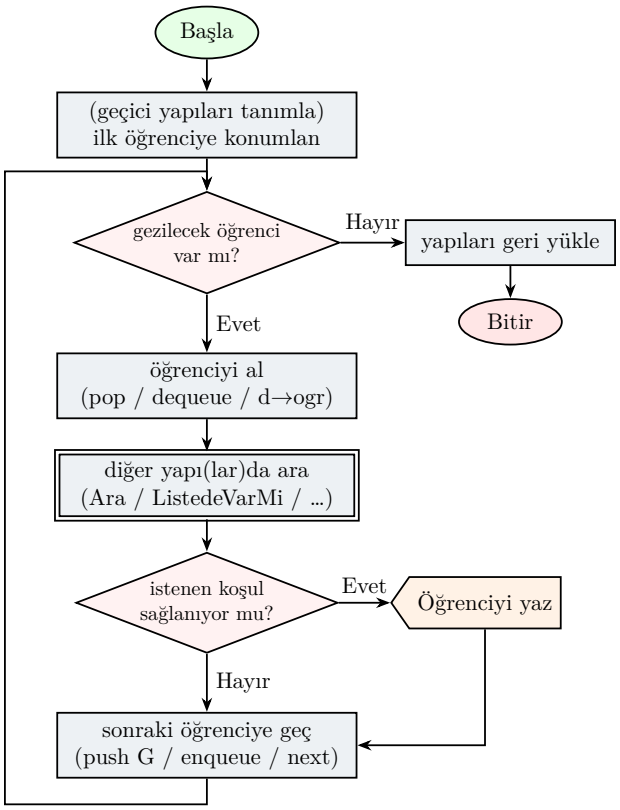
İstenen	Mantık
Sadece A	A'yı gez; B'de (ve C'de) *yoksa* yaz
Hem A hem B	A'yı (küçük/sırasız olanı) gez; B'de *varsa* yaz (ağaçta aramak ucuzdur)
Üçünü de	A'yı gez; B'de *ve* C'de varsa yaz
Sadece bir ders	her yapıyı ayrı gez; diğer ikisinin hiçbirinde yoksa yaz
Tam iki ders	her öğrenci için kaç yapıda olduğunu say (sayaç == 2); tekrar yazmamaya dikkat
En az iki (birden fazla / "çok")	A'yı gez: B veya C'de varsa yaz; sonra A'da olmayan ama B ve C'de olanlar

8.3 Şablon (hocanın tahtadaki çözümüne göre)

```
// Y: yigin (Matematik), K: kuyruk (Turkce), T: AVL (Hayat Bilgisi, numaraya gore)
// Istenen: uc dersi de alanlar
UcDersiAlanlar(Stack Y, Queue K, AVL T)
   Stack Z                                  // gecici yigin (tanim ihmal)
   while (Y.isEmpty() == false)
      A = Y.pop();  Z.push(A)               // yigindan al, kaybetme
      if (T.Find(A.no) != NULL)             // AVL'de ara: log n
         n = K.size()
         for (j = 1; j <= n; j++)           // kuyrugu bir tur dondur
            B = K.dequeue()
            if (A.no == B.no)  Yaz(A)
            K.enqueue(B)                    // ayni sirayla geri koy
   while (Z.isEmpty() == false)  Y.push(Z.pop())   // yigini eski haline getir
```

Akış diyagramı: genel farz edin ki fonksiyonu

Bir yapıyı gez (yığın: geçici yığına aktararak, kuyruk: döndürerek, liste: `next` ile), her öğrenciyi diğer yapı(lar)da ara, koşul sağlanırsa yaz, sonunda yapıları geri yükle. Yığın ve kuyruk gezme diyagramları Bölüm 5'tedir.



8.4 Cevap kâğıdında olması gerekenler

1. **Varsayımlar:** hangi yapıda hangi ders, ağacın hangi alana göre indeksli olduğu, "öğrenci aynı = numara aynı" kabulü, "çok ders = en az iki" gibi yorumlar.
2. **Kaba kod:** satır numaralı, girintili; sadece temel işlemler. Yardımcı fonksiyon yazdıysanız (ör. `KuyruktaVarMi`) onun kodunu da verin.
3. **Mantık:** her bloğun ne yaptığını madde madde açıklayın (*"herşeyi ayrıntılı açıklama olarak istiyor yoksa puan vermez... köşelere akış diyagramlarında vs hepsi için geçerli"*).
4. **Yapıların korunması:** yığın/kuyruk için geçici yapı veya döndürme; ağaçtan silme yaptıysanız (ör. B+'tan geçici silme) sonunda geri ekleyin.
5. **Akış diyagramı:** başla, döngü koşulu (karar), işlem kutuları, fonksiyon çağrısı (çift kenar), ekrana yazma (göz sembolü), dönüş okları, bitir.
6. **Yürütme zamanı:** değişkenleri tanımlayın (a: AVL'deki, y: yığındaki, k: kuyruktaki öğrenci sayısı…), satır satır tekrar sayısı tablosu, toplam $T(\cdot)$, en büyük dereceli terim $\Rightarrow O(\cdot)$; en iyi/en kötü durumun ne zaman oluştuğu.

Yapılmaması gerekenler

- Yığına/kuyruğa indisle erişmek (`stack[i]`, `queue.pop[j]`), `push` ile veri okumak.
- "`ogrsayi`" gibi bilinmeyen boyutları sabit kabul etmek (gerekirse `size()` veya `isEmpty()` döngüsü kullanın).
- Yapıyı boşaltıp geri doldurmamak (Not2 ihlali).
- Klasördeki "Farz edin ki Soruları.pdf" kodlarını ezberlemek (hatalılar; hoca da *"ezber farz edin ki kodlarını biliyor"*).
- Okunaksız, düzensiz sayfa: *"sayfa düzeni olmalı yoksa gene okumuyor karışık olduğunu düşünürse"*.

Tüm çıkmış farz edin ki sorularının tam çözümleri: `02_Vize_Cozumleri.pdf` (2020, 2021, 2024, 2025, bilinmeyen yıl, örnek sorular).

9 Ek: B+ Ağaçları ve Graf Arama (bazı yıllarda vizeye girdi)

2020 ve 2024 vizelerinde B+, 2024 vizesinde ve 2024–25 mazeret sınavında graf (BFS/sonrakök) soruldu. Hoca konuları hızlı işlerse vizeye girer.

9.1 B+ ağacı

- Çocuk sayısı 3 ve daha fazla olan çok yollu dengeli ağaç (*M-ary*); çocuk sayısı arttıkça yükseklik $\log_M n$'e düşer, disk erişimi azalır.
- **Bütün veriler yapraklardadır**, yapraklar aynı seviyededir; iç düğümler sadece yol gösteren *anahtarları* tutar. Bir iç düğümdeki i. anahtar, $i+1$. alt ağacın en küçük verisidir (*"sağdakinin minimumu yukarı anahtar olarak kopyalanır"*).
- **Hocanın formülleri** (M: iç düğüm çocuk sayısı, L: yaprak parametresi; aksi söylenmezse $L = M$):

	En fazla	En az
İç düğüm anahtar	$M-1$	$\lceil M/2 \rceil - 1$
Yaprak veri	$L-1$	$\lceil L/2 \rceil - 1$
Kök	$M-1$	1 (kökün "süperliği")

  Soru dilinden çeviri: "iç düğüm çocuk sayısı 4" $\Rightarrow M = 4$; "iç düğüm anahtar sayısı 3" $\Rightarrow M = 4$; "yapraklarda tutulan maksimum veri 4" $\Rightarrow L = 5$ (2025 final için WhatsApp'ta da M=4, L=5 sonucuna varılmış).
- **Ekleme:** yaprağa sıralı eklenir. Yaprak taşarsa (L veri): sola $\lfloor L/2 \rfloor$ en küçük, sağa kalanlar; *sağdakinin en küçüğü yukarı kopyalanır*. İç düğüm taşarsa (M anahtar): sola $\lceil M/2 \rceil - 1$ anahtar, $\lceil M/2 \rceil$. anahtar yukarı *taşınır*, kalanlar sağa. Kök bölünürse yeni kök oluşur.
- **Silme:** veri yapraktan silinir. Yaprak minimumun altına düşerse *sağ kardeşiyle* (en sağdaki yaprak ise sol kardeşiyle) **birleştir–böl** (veriler iki yaprağa yeniden paylaştırılır); bu da minimumu sağlayamıyorsa iki yaprak **birleştirilir** ve üstteki anahtar silinir (iç düğüm de aynı mantıkla düzeltilir). Anahtarlar her zaman "sağ alt ağacın en küçüğü" olacak şekilde güncellenir. *"B şıkkını yanlış yaparsanız C şıkkı da otomatikman yanlış olacak."*

Örnek: $M = L = 4$ ile K, L, M, N, O, P, Q, R, S, T ekle; N ve O sil

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Parametreler: iç düğüm çocuk sayısı $M = 4 \Rightarrow$ iç düğümde en fazla $M-1 = 3$, en az $\lceil M/2 \rceil - 1 = 1$ anahtar; $L = 4 \Rightarrow$ yaprakta en fazla $L-1 = 3$, en az $\lceil L/2 \rceil - 1 = 1$ veri.

Başlangıç ağacı

∅

K ekle: K yaprağa eklendi (yaprakta boş yer vardı).

K

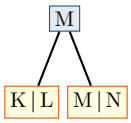
L ekle: L yaprağa eklendi (yaprakta boş yer vardı).

K | L

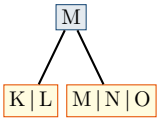
M ekle: M yaprağa eklendi (yaprakta boş yer vardı).



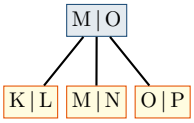
N ekle: N eklenince yaprakta 4 veri oldu (maks. 3) ⇒ yaprak bölünür: sola $\lfloor 4/2 \rfloor$=2 en küçük (K, L), sağa 2 (M, N); sağdakinin en küçüğü M yukarı anahtar olarak KOPYALANIR. Kök bölündüğü için yeni kök oluştu: [M].



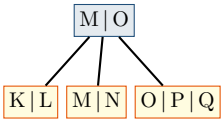
O ekle: O yaprağa eklendi (yaprakta boş yer vardı).



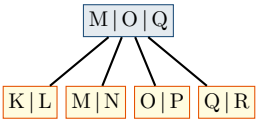
P ekle: P eklenince yaprakta 4 veri oldu (maks. 3) ⇒ yaprak bölünür: sola $\lfloor 4/2 \rfloor$=2 en küçük (M, N), sağa 2 (O, P); sağdakinin en küçüğü O yukarı anahtar olarak KOPYALANIR.



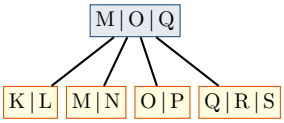
Q ekle: Q yaprağa eklendi (yaprakta boş yer vardı).



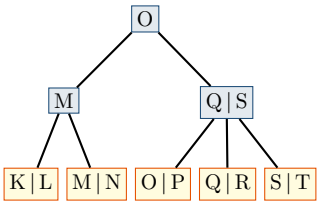
R ekle: R eklenince yaprakta 4 veri oldu (maks. 3) ⇒ yaprak bölünür: sola $\lfloor 4/2 \rfloor$=2 en küçük (O, P), sağa 2 (Q, R); sağdakinin en küçüğü Q yukarı anahtar olarak KOPYALANIR.



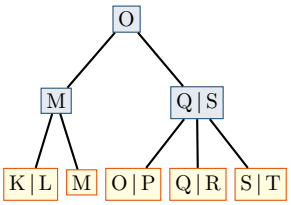
S ekle: S yaprağa eklendi (yaprakta boş yer vardı).



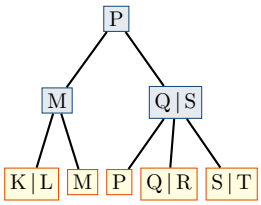
T ekle: T eklenince yaprakta 4 veri oldu (maks. 3) ⇒ yaprak bölünür: sola $\lfloor 4/2 \rfloor$=2 en küçük (Q, R), sağa 2 (S, T); sağdakinin en küçüğü S yukarı anahtar olarak KOPYALANIR. İç düğümde 4 anahtar oldu (maks. 3) ⇒ iç düğüm bölünür: sola $\lceil 4/2 \rceil - 1$=1 anahtar (M), O yukarı TAŞINIR, sağa Q, S. Kök bölündüğü için yeni kök oluştu: [O].



N sil: N yapraktan silindi. Minimum kural sağlanıyor; gerekirse üst anahtarlar sağdakinin en küçüğü ile güncellenir.



O sil: O yapraktan silindi. Minimum kural sağlanıyor; gerekirse üst anahtarlar sağdakinin en küçüğü ile güncellenir.



Zaman ve alan karmaşıklığı

İşlem	En iyi	En kötü	Ek alan	Not
Arama	$\Omega(\log_M n)$	$O(\log_M n)$	$O(1)$	kökten yaprağa bütün yollar eşit uzunlukta (yapraklar aynı seviyede)
Ekleme	$\Omega(\log_M n)$	$O(\log_M n)$	$O(1)$	bölünme köke kadar yayılabilir; her seviyede sabit iş
Silme	$\Omega(\log_M n)$	$O(\log_M n)$	$O(1)$	birleştir–böl / birleştirme köke kadar yayılabilir
Sıralı gezme	$\Theta(n)$	$\Theta(n)$	$O(1)$	yapraklar soldan sağa okunur

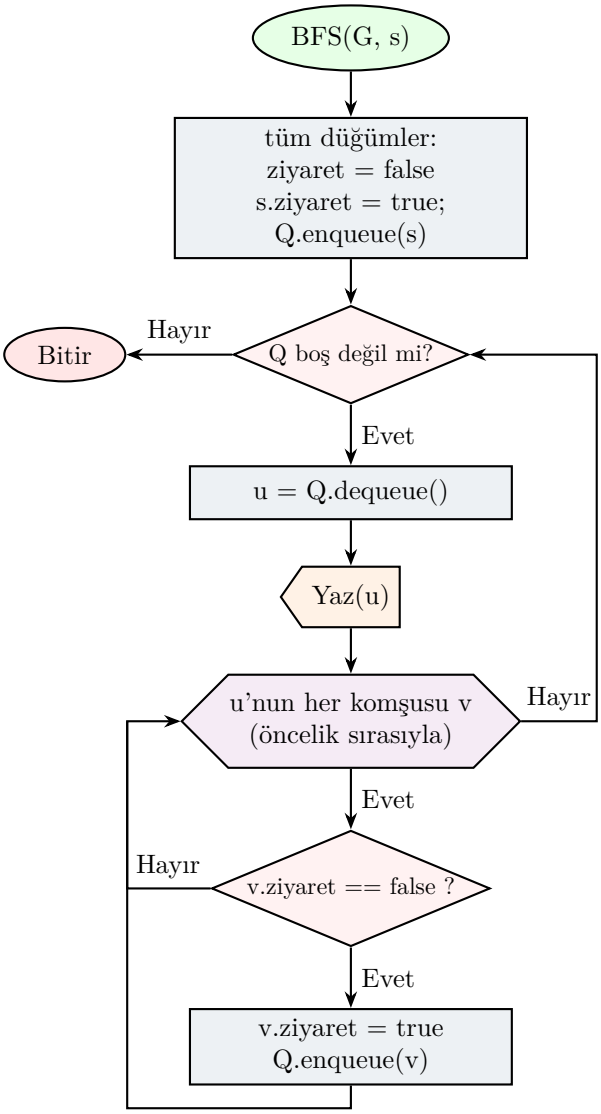
9.2 Graflar: BFS, DFS ve sonrakök

- Gösterim: komşuluk matrisi (n^2 yer) veya komşuluk listesi (seyrek graflarda tercih edilir).
- **Genişlik öncelikli arama (BFS):** başlangıç kuyruğa konur; kuyruktan çıkan düğümün ziyaret edilmemiş komşuları (soruda verilen alfabetik önceliğe göre) kuyruğa eklenir. Ağırlıksız grafta en kısa yolları (kenar sayısı) verir. $O(V + E)$.
- **Derinlik öncelikli arama (DFS):** yığın / özyineleme; bir komşuya gidip olabildiğince derine inilir. Öncekök sırası = ilk ziyaret sırası; **sonrakök sırası = bir düğümün bütün komşuları bitince yazılması**.
- **Öncelik notuna dikkat:** “alfabetik olarak önde olanı önce” veya “sonda olanı önce” (2024 vize, mazeret, 2020 büt) – komşuluk listesini buna göre sıralayın.

Akış diyagramı: genişlik öncelikli arama (BFS) – kuyruk ile

```
BFS(G, s)
  her u icin u.ziyaret = false
  s.ziyaret = true;  Q.enqueue(s)
  while (!Q.isEmpty())
     u = Q.dequeue();  Yaz(u)
     for (her komsu v, oncelik sirasiyla)
        if (v.ziyaret == false)
           v.ziyaret = true
           Q.enqueue(v)
```

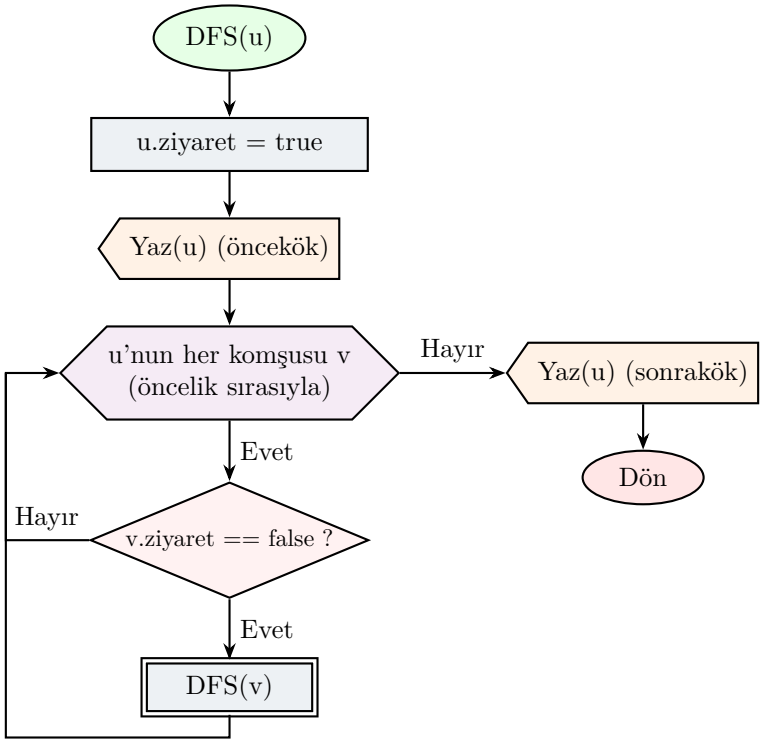
Komşu önceliği soruda verilir (alfabetik önde/sonda). 2025–26 finalinde BFS akış diyagramı ve karmaşıklığı soruldu.



Akış diyagramı: derinlik öncelikli arama (DFS) ve sonrakök sırası

```
DFS(u)
  u.ziyaret = true
  Yaz(u)            // oncekok sirasi
  for (her komsu v, oncelik sirasiyla)
     if (v.ziyaret == false)
        DFS(v)
  Yaz(u)            // sonrakok sirasi
```

Özyineleme arka planda yığın kullanır. “Düğümleri sonrakök mantığına göre sıralayınız” sorusunun cevabı ikinci `Yaz` satırının ürettiği sıradır.



Zaman ve alan karmaşıklığı

İşlem	En iyi	En kötü	Ek alan	Not
BFS (komşuluk listesi)	$\Theta(V+E)$	$\Theta(V+E)$	$O(V)$	her düğüm bir kez kuyruğa girer, her kenara bir kez bakılır
DFS / sonrakök	$\Theta(V+E)$	$\Theta(V+E)$	$O(V)$	özyineleme (yığın) derinliği en fazla V
Komşuluk matrisiyle	$\Theta(V^2)$	$\Theta(V^2)$	$O(V^2)$	her düğüm için bütün satır taranır

Örnek (ders slaytındaki graf, A’dan, alfabetik önde öncelikli)

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Komşuluk listesi (komşular alfabetik olarak ÖNDE olan önce sıralanmış):

A: C, D, E

B: –

C: A, E

D: A, E

E: A, C, D

Genişlik öncelikli arama (BFS) — kuyruk kullanılır:

Çıkan	Komşular	Kuyruğa eklenen	Kuyruk
A	C, D, E	C, D, E	C, D, E
C	A, E	–	D, E
D	A, E	–	E
E	A, C, D	–	boş

BFS ziyaret sırası: A, C, D, E

Uzaklıklar (kenar sayısı): A=0; C=1; D=1; E=1

Derinlik öncelikli gezme ve sonrakök (postorder) — bir düğümün bütün (ziyaret edilmemiş) komşuları bitince düğüm listeye yazılır (yığın mantığı):

A ziyaret edildi, yığın: [A]

C ziyaret edildi, yığın: [A, C]

E ziyaret edildi, yığın: [A, C, E]

D ziyaret edildi, yığın: [A, C, E, D]

D **bitti** ⇒ listeye yaz; yığın: [A, C, E]

E **bitti** ⇒ listeye yaz; yığın: [A, C]

C **bitti** ⇒ listeye yaz; yığın: [A]

A **bitti** ⇒ listeye yaz; yığın: []

DFS (öncekök) ziyaret sırası: A, C, E, D

Sonrakök (postorder) sırası: D, E, C, A

Not: B düğüm(ler)ine A'den yol yoktur, bu yüzden listeye girmez.